SPRINGER BRIEFS IN COMPUTER SCIENCE

Bo Zhao · Byung Chul Tak
Guohong Cao

Mobile Web Browsing Using the Cloud

Springer

www.allitebooks.com

SpringerBriefs in Computer Science

Series Editors

Stan Zdonik
Peng Ning
Shashi Shekhar
Jonathan Katz
Xindong Wu
Lakhmi C. Jain
David Padua
Xuemin Shen
Borko Furht
V. S. Subrahmanian
Martial Hebert
Katsushi Ikeuchi
Bruno Siciliano

For further volumes:
http://www.springer.com/series/10028

www.allitebooks.com

Bo Zhao · Byung Chul Tak
Guohong Cao

Mobile Web Browsing Using the Cloud

Springer

www.allitebooks.com

Bo Zhao
Byung Chul Tak
Guohong Cao
Department of Computer Science and Engineering
The Pennsylvania State University
University Park, PA
USA

ISSN 2191-5768 ISSN 2191-5776 (electronic)
ISBN 978-1-4614-8102-7 ISBN 978-1-4614-8103-4 (eBook)
DOI 10.1007/978-1-4614-8103-4
Springer New York Heidelberg Dordrecht London

Library of Congress Control Number: 2013941911

References to various copyrighted trademarks, servicemarks, marks and registered marks owned by the respective corporations and/or connected subsidiaries may appear in this book. We use the names, logos, and images only in an editorial fashion with no intention of infringement of the trademark.

© The Author(s) 2014
This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed. Exempted from this legal reservation are brief excerpts in connection with reviews or scholarly analysis or material supplied specifically for the purpose of being entered and executed on a computer system, for exclusive use by the purchaser of the work. Duplication of this publication or parts thereof is permitted only under the provisions of the Copyright Law of the Publisher's location, in its current version, and permission for use must always be obtained from Springer. Permissions for use may be obtained through RightsLink at the Copyright Clearance Center. Violations are liable to prosecution under the respective Copyright Law. The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

www.allitebooks.com

Preface

Smartphones have become a key element in providing greater user access to the mobile Internet. Many complex applications which used to be limited to PCs, have been developed and operated on smartphones. These applications extend the functionalities of smartphones, making them more convenient for users to be connected. However, they also greatly increase the power consumption of smartphones, making users frustrated with long delays in Web browsing. This book surveys existing techniques to address these problems, and shows that one key reason for the delays and high power consumption is due to the local computational limitation at the smartphone (e.g., running java scripts or flash). To address this issue, we introduce an architecture called Virtual Machine-based Proxy (VMP), to shift the computing from smartphones to the VMP which may reside in the cloud. The book illustrates the feasibility of deploying the proposed VMP system in 3G networks through a prototype using Xen virtual machines (in cloud) and Android Phones with ATT UMTS network. This book also includes cloud techniques to address scalability issues, resource management techniques to optimize the performance of the VMs on the proxy side, compression techniques to further reduce the bandwidth consumption, and adaptation techniques to address poor network conditions at the smartphone.

March 2013

Bo Zhao
Byung Chul Tak
Guohong Cao

www.allitebooks.com

Contents

1 Introduction 1

2 Related Work 5
2.1 Existing Work on Power Saving 5
2.2 Thin-Client Based Approach 6
2.3 Proxy Based Approach 7
2.4 Cloud Based Approach 8

3 Motivation 11
3.1 Delay Issues in Web Browsing 11
3.2 Power Consumption of Web Browsing in 3G 14

4 Virtual Machine Based Proxy 17
4.1 System Architecture 17
4.2 Communication Mechanisms 19
4.2.1 Features for Opening Web Pages 19
4.2.2 Supporting User Interactions 19
4.2.3 Compressions 20
4.2.4 Adaptation Techniques 20
4.3 Design of VMP at the Service Provider 20
4.3.1 VM Management 20
4.3.2 Managing VM Image 22
4.3.3 Scalability Issues 22
4.3.4 Implementation Issues 25
4.4 Security and Privacy Issues 26
4.4.1 Security-Enhanced Model for the VMP Architecture 27
4.4.2 Asymmetric Encryption and Authentication System 28
4.4.3 Trusted Computing Module 29
4.4.4 Antivirus Scanning Service 30
4.4.5 System Overhead 30
4.5 Discussions 31
4.5.1 Supporting Interactive Applications 31
4.5.2 Deployment Issues 32

www.allitebooks.com

5 Performance Measurements 33
5.1 The Experimental Setup 33
5.2 The Web Page Benchmark 34
5.3 Experimental Results on Delay 35
5.3.1 Average Delay for Opening Benchmark Web Pages 36
5.3.2 Effects of Transmitting Multiple Screens of Data 38
5.3.3 Effects of Data Compression 39
5.4 Experimental Results on Power Consumption 40
5.4.1 Power Consumption of the Smartphone Browser 41
5.4.2 Power Consumption with Web Page Benchmark. 45
5.4.3 The Benefit of Performing Antivirus Scanning with VMP 47

6 Conclusions and Future Research Directions. 49

References 51

www.allitebooks.com

Chapter 1
Introduction

Abstract This chapter introduces the technical challenges arising from supporting web browsing on smartphones. For ease of understanding and further discussions, we briefly describe existing techniques on reducing the access delay and power consumption of smartphones. Then, we propose a new architecture, called *Virtual Machine-based Proxy (VMP)*, which aims to shift the computationally intensive web browsing tasks from smartphones to reduce the delays and power consumption. Finally, we highlight the advantages of the VMP architecture and provide detailed organization of the book.

Due to the arrival in the market of a new generation of improved smartphones, with large screens, improved user interfaces, and advanced features such as GPS, smartphones have become a key element in providing greater user access to the mobile Internet. For several years, the demand for smartphones has outpaced the remainder of the mobile phone market. Approximately half of the U.S. mobile consumers own smartphones and could account for nearly 70 % of all U.S. mobile devices in 2013 [1].

Different from traditional mobile phones, smartphones incorporate mobile operating systems such as Google's Android, Apple's iOS, or others, leading to more capability than traditional mobile phones. Many complex applications which used to be limited to Personal Computers (PCs), have been developed and operated on smartphones. These applications extend the functionalities of smartphones, making them more convenient for users to connect to the Internet; however, the trade-off for greater capability is a greatly increased workload leading to increased power consumption.

Smartphones, powered by battery, create a major concern for the battery's life in the mobile communication industry. Unfortunately, the rate at which battery performance improves has been much slower than the development of the devices' capabilities [2]. Aside from major breakthroughs, significant improvement in the foreseeable future is doubtful. Rather than improving the amount of energy a power source contains, this research explores an alternative: carefully designing

B. Zhao et al., *Mobile Web Browsing Using the Cloud*,
SpringerBriefs in Computer Science,
DOI: 10.1007/978-1-4614-8103-4_1, © The Author(s) 2014

www.allitebooks.com

communication protocols that allow smartphones to perform expected functions and the similar services while minimizing overall power consumption. Likewise, to respond to users' frustrations from long delays during web browsing with smartphones, improved design to protocols reduce the impediment.

Some research has focused on optimizing batteries' power [3–5]; however, limitation of these studies is for reducing power consumption of one component of the smartphone such as the display, the wireless interface [5], or the network protocol. From the web server's perspective, smartphones receive the same consideration as other computing devices such as desktop computers in offices despite a fundamental difference between smartphones and desktop PCs such as display size and power source. To address the fundamental difference between smartphones and desktop PCs, some web servers adopt transcoding techniques [6, 7], which treat smartphones differently from other computers. For example, if the transmitted request originates from a smartphone, servers reduce resolution of images to match the smartphone and disable some javascripts to conserve the smartphone's energy. To use such techniques, redesign of the web server is necessary, and consequently, can significantly increase costs which only a limited number of web servers are willing to absorb. Some other techniques [8, 9], based on optimizations at the *proxy*; i.e., a computer system that acts as an intermediary for requests from clients seeking resources from servers. For example, the flash proxy [8] translates flash to another content format on a proxy. Opera Mini [9] also relies on proxies and multiple transcoding approaches to reduce the traffic load of web browsing on smartphones.

Even with these transcoding techniques, they do not adequately address the problems of long delays and high power consumption, because 3G networks have greater bandwidth compared to 2G or 2.5G networks, and bandwidth, although an important factor, is not necessarily the bottleneck for most web pages. In fact, our study revealed that the workload for smartphone's CPU can easily reach 100 % for an extended period during web browsing rendering local computational capability the real bottleneck for most web pages. As a result, to address the problems of long delays and high power consumption, design of a new architecture for smartphones to access the mobile Internet is necessary.

This book, as shown in Fig. 1.1, proposes a new architecture to shift the intensive computations of web browsing from smartphones to accomplish a reduction in delays and power consumption. In this architecture, a proxy is added between the web server and the smartphones, and a new client is added to the smartphone to interact with the proxy. For smartphones, instead of sending a request to the web server, the request arrives at the proxy through the wireless network. The proxy sends the request to the web server and receives a reply. Then, the proxy sends the reply to the smartphone through the wireless network. At this time, the proxy can run different optimization techniques to reduce consumption of resources within the smartphone. For example, the proxy can act as an actual web browser, which displays the content of the web page. Instead of sending the original HTTP reply which contains many CPU intensive objects, the proxy transmits a screen copy of browser to the smartphone. The proxy has other adaptive capabilities based on the level of energy remaining in the battery and the wireless signal strength of the smartphone. The proxy can be

www.allitebooks.com

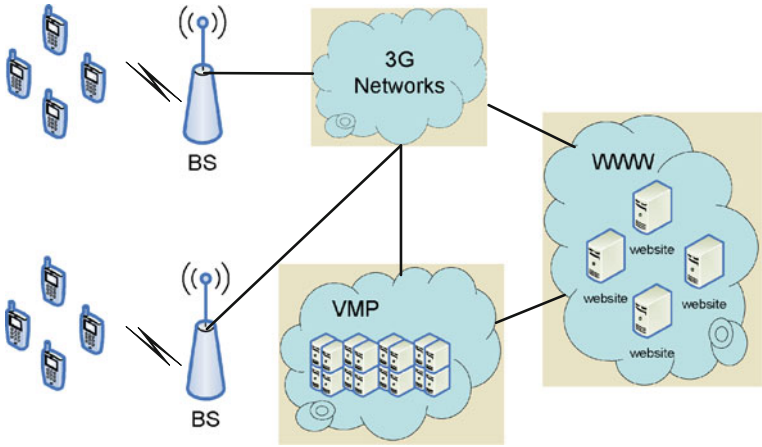

Fig. 1.1 The VMP architecture. A web request from the smartphone arrives at the proxy, which forwards the request to the web server and receives a reply. Then, the proxy copies the screen and sends the data to the smartphone through the wireless network

anywhere in the Internet, but can achieve better performance if residing inside the telecommunication's network. Due to security and privacy issues, the proxy initiates a Virtual Machine (VM) for each smartphone request and releases the resource after serving the request.

Different from existing works on proxy and transcoding [6–8], which are limited to dealing with images and multimedia content, the architecture of our proxy based approach shifts computing from smartphones to the proxy's VMs, thus, the architecture is, appropriately, a *VM-based Proxy (VMP)*. Through the VMP architecture, dynamic content (javascript, flash, etc.) operates at the server side, and consequently conserves the smartphone's energy reserves.

To illustrate the feasibility of deploying the proposed VMP architecture in 3G networks, we built a prototype using Xen virtual machines and Android Phones with T-Mobile UMTS network. Experimental results show that compared to a normal smartphone browser, the VMP approach reduces delay by more than 80 % and reduces power consumption during web browsing by more than 45 %, on average. For browsing content-rich commercial web pages, the VMP approach can reduce the delay by more than 92 %, and reduce the power consumption of web browsing by more than 58 %.

The organization of the remainder of this book is: A discussion of related work in Chap. 2, presentation of motivation for the study in Chap. 3, description of the design of the VM-based proxy in Chap. 4, and introduction of the testbed and presentation of the experimental results in Chap. 5. Finally, conclusions and future research directions are presented in Chap. 6.

www.allitebooks.com

Chapter 2
Related Work

Abstract This chapter describes some related research similar to that appearing in this book. Since much of previous study is dated and designed for general mobile devices rather than modern smartphones, the initial discussion centers on research for conserving mobile devices' energy. Then, we survey techniques following three different approaches to offload the computations of mobile devices; that is, thin-client based approach, proxy based approach, and cloud based approach. For each approach, we compare it with our VMP approach and highlight the main differences.

2.1 Existing Work on Power Saving

Recently, research for reducing power consumption of smartphones has received considerable attention. Among the various components that contribute to power consumption of mobile devices, screen display ranks highest, easily consuming half of total available energy. Simply reducing the brightness or refreshing frequency is not desirable due to negative impact on the user's experience. To address this problem, Iyer et al. [10] proposed to conserve energy by matching the display's consumption to the functionality required by the workload. Zhong et al. [3] studied the impact of human factors on energy efficiency and proposed a cache solution to reduce the power consumed by the screen display.

Since more and more applications (such as web browsing, speech recognition) are computationally intensive, the CPU workload can easily reach 100 % for an extended period of time. It is known that the CPU can consume as much as half of the power consumed by the 3G wireless transmission. To address this problem, Flinn et al. proposed power management solutions [4] through adaptations which dynamically modify application behavior to conserve energy and allow the operating system to manage battery resources to extend the battery life, using feedback and application history. Turduken et al. [11] proposed an approach to integrate a series of components that operate at various power levels to extend the battery life. The IBM Wristwatch

B. Zhao et al., *Mobile Web Browsing Using the Cloud*,
SpringerBriefs in Computer Science,
DOI: 10.1007/978-1-4614-8103-4_2, © The Author(s) 2014

Computer [12] provided hardware-level tradeoffs and kernel optimizations to save power by using various standby and idle configurations. Havinga et al. [13] reviewed a number of energy reduction techniques such as minimizing capacitance, avoiding unnecessary and wasteful activity, and reducing voltage and frequency in the design of portable devices. However, all these studies focus on optimizing power use of individual components of the mobile device, which is different from our work which focuses on reducing the power consumption of web browsing.

2.2 Thin-Client Based Approach

A system based on the thin-client consists of a server and a client that communicate over a network using a remote display protocol. The protocol allows visualization of graphic displays at the client's device across the network, while execution of the application occurs on the server. Using the remote display protocol, the client transmits user inputs to the server which returns screen updates from the application to the client [14].

To offload local computing, many approaches use the thin-client, such as Sun Ray [15], Microsoft's Remote Desktop [16], Virtual Network Computing [17], X window system [18] and THINC [14]. THINC is a thin-client protocol, currently at the research stage. It introduces a simple virtual display driver that intercepts drawing commands at the device layer, packetizes and sends the commands over the network to the client's display. Several studies examined the web browsing performance of thin-client based approaches [19, 20], which used the technique to improve the performance of web browsing on wireless PDAs and appeared first in [19]. pTHINC [20] has been implemented in the X/Linux environment and its performance has been compared against widely used commercial approaches. These studies focus on methods for deploying THINC [14] to PDA devices and provide video services. However, the design of the thin-client based approach's original intent was to offload local computing by using increased network traffic to reduce the effects of network delay on users' experiences. Therefore, the thin-client based approach's design does not include considerations for power efficiency, and existing research only focuses on reducing delays in high speed wireless interfaces such as WiFi, characterized by short latency and high bandwidth. Although both the VMP-based approach and thin-client based approach employ a function for copying screens, the proposed VMP architecture addresses issues of power conservation in 3G, which has long latency and slow data transfers compared to WiFi. Moreover, the proposed VMP approach provides dynamic adaptation, security protection, and cloud based service management.

2.3 Proxy Based Approach

Many research projects and commercial systems used proxies to adapt content to better accommodate the special characteristics of mobile devices [7, 8, 21]. For example, Digestor [7] dynamically adjusts the requested web pages to coincide with a given PDA's display. Lara et al. [22] proposed a proxy-based approach to enable collaboration and document editing on bandwidth-limited mobile devices. Flashproxy [8] addressed issues for supporting flash on smartphones which lack necessary software or physical resources. Flashproxy supports flash by replacing it with an AJAX-based remote display component, but Flashproxy does not address the performance issue of web browsing.

Transcoding techniques can transform an image object [23] or video object [24] from one version to another and usually occurs when the mobile device does not support the current format or has limited storage capacity, which mandates reduced file size or conversion of incompatible or obsolete data to a better-supported format. Many proxies rely on transcoding techniques to support multimedia services on mobile devices because of the limited bandwidth and hardware resources. For example, Fox and Brewer [21] introduced a real-time distillation and refinement approach to decrease the quality of the image of web pages to reduce the latency and bandwidth requirements of PDAs. Han et al. [6] introduced image transcoding proxy techniques to support web browsing services on mobile devices.

The aforementioned works on transcoding and proxy are limited to deal with image and multimedia content. At that time (5–10 years ago), dynamic web content such as flash and javascripts were not very popular, which is a different case today. Simply disabling dynamic web content may not be sufficient, since many useful flash or javascripts will not function and some useful content may be invisible. Through the proposed VMP architecture, the dynamic web content such as flash and javascript operate from the proxy, and only the display data arrives at the smartphone. In other words, the VMP architecture improves the performance of web browsing by shifting the computing workload of dynamic web content to the remote proxy.

Although many proxy-based techniques such as Opera Mini [9] or Amazon Kindle [25] can improve the performance of web browsing, these techniques are different from that proposed in the current study. Opera Mini converts the web page to a different format which can reduce the page size to fit smartphones' screens. Although Opera Mini can reduce delay, compared to the original web browser, the smartphone still needs to parse the data and use its rendering engine to layout the content. The proposed VMP approach offloads all computations to the proxy, and only compressed screen images arrive at the smartphone. As a result, this approach can provide better performance in terms of power consumption and delay compared to Opera Mini. Moreover, Opera Mini disables some javascripts after loading the web page, thereby affecting some characteristic of the display, which the VMP architecture addresses. Kindle Fire's Silk browser is similar to Opera and consequently different from VMP.

In our previous work [28], we have presented a proxy-based approach to reduce the delay and power consumption of web browsing on smartphones. This book is

based on our previous work, but also includes cloud techniques to address scalability issues, resource management techniques to optimize the performance of the VMs on the proxy side, and techniques to address security and privacy issues.

2.4 Cloud Based Approach

Smartphones have strict resource constraints in terms of computation, memory, and energy. As a result, smartphones cannot operate many computationally intensive applications. To address the computational limitations of smartphones, Chun and Maniatis [26] proposed a cloud-based approach for offloading some computationally intensive tasks to computers in the cloud, where they are executed in a cloned whole-system image of the smartphone. This augmented execution overcomes the limitations of smartphones' hardware and is automatic for applications whose developers need few or no modifications to their applications. Some augmentations can operate in the background, for asynchronous operations such as periodic file scans. For synchronous operations intrinsic to the application (e.g., a series of floating-point instructions in the application code), augmentation occurs by blocking the program progress on the smartphone until the result arrives from the cloud. For concurrent operations (e.g., taint-checking), augmentation can also be concurrent in the cloud or even speculative with the ability to undo operations on the smartphone according to the result from the cloud.

To reduce the latency between smartphones and servers, Cloudlet [27] uses a nearby remote server and thin-client techniques to virtually extend the computing capability of a smartphone, allowing support for computationally intensive applications. In this architecture, a cloudlet is a trusted, resource-rich computer or a cluster of computers, well-connected to the Internet and available for use by nearby smartphones. This approach leverages a transiently customized, proximate infrastructure as a smartphone moves with its user from location to location. The interactive response, which is essential for seamless augmentation of human cognition, occurs easily in this architecture because of the cloudlet's physical proximity and one-hop network latency. Using a cloudlet also addresses the challenge of meeting the peak bandwidth demand of multiple users interactively generating and receiving large amounts of content such as high-definition video and high-resolution images.

MAUI [30] enables fine-grained energy-aware offloading of mobile code to the infrastructure to save power and maximizes the potential for energy saving through fine-grained code offloading while minimizing the changes required for applications. To achieve this goal, first, MAUI uses code portability to create two versions of an application for smartphones, one of which runs locally on the smartphone and the other runs remotely in the infrastructure. Managed code enables MAUI to ignore the difference in the instruction set between the smartphones and servers. Second, MAUI uses programming reflection combined with type safety to automatically identify the remote, executable methods and extract only the program state needed by those methods. Third, MAUI profiles each method of an application and uses serialization

to determine its shipping cost through the network. MAUI combines the network and CPU cost with measurements of the wireless connectivity, such as its bandwidth and latency to formulate code offloading as a linear programming problem. Based on the solution to this problem, partitioning the application at runtime maximizes energy saving under the current networking condition. Since serialization can occur at runtime, MAUI's application profiling operates continuously to provide up-to-date estimates of each method's cost. The combination of code portability, serialization, reflection, and type safety allows MAUI to offer dynamic fine-grained offloading of code with minimal burden on the programmer.

These cloud-based approaches [26, 27, 30] mainly focus on future computationally intensive applications (e.g., natural language processing, face recognition, etc.) which are rarely used currently. However, our work focuses on shifting computationally expensive web browsing applications, such as flash or javascript, to remote infrastructures to reduce delays and conserve power. Since web browsing is widely common, the benefits from our technique are broad and pervasive for users.

Chapter 3
Motivation

Abstract This chapter describes the motivations for employing the VMP approach to reduce delays and power consumption attributable to web browsing on smartphones. In 3G networks, the bandwidth has significantly increased compared to earlier generations, but opening web pages on smartphones remains slow and consumes significant energy. The current research has identified two major reasons for these phenomena through experiments. First, today's web pages have become more complex, requiring greater computational power for processing. Second, the bottleneck for web browsing has shifted from network bandwidth to CPU in 3G networks. As a result, the proposed VMP approach which offloads computation of web browsing to the cloud, is an appropriate solution.

3.1 Delay Issues in Web Browsing

To understand the reason for the extended time to open a web page on smartphones, consideration must include the design of modern web browsers. Today's web browser is quite complex since it has to support the processing of various scripts, such as JavaScript, embedded in HTML documents. Further, web pages must process Cascading Style Sheets (CSS), which describe the semantics of the presentation and the style rules of a web page such as the layout, color and fonts. The Document Object Model (DOM) is an interface that allows programs and scripts to update the content, structure and style of HTML documents. After the HTML code has been parsed, the nodes in the DOM tree store the HTML data. After the CSS code has been parsed, the style and layout properties are assigned to these nodes in the DOM tree. Then the web browser can display them on the screen. After the web browser receives the main HTML page, the data transmission mainly comes from content objects, such as HTML files, JavaScript files, images, and flashes, which have URL references. Hence, the web browser needs to retrieve and add them to the DOM tree.

B. Zhao et al., *Mobile Web Browsing Using the Cloud*,
SpringerBriefs in Computer Science,
DOI: 10.1007/978-1-4614-8103-4_3, © The Author(s) 2014

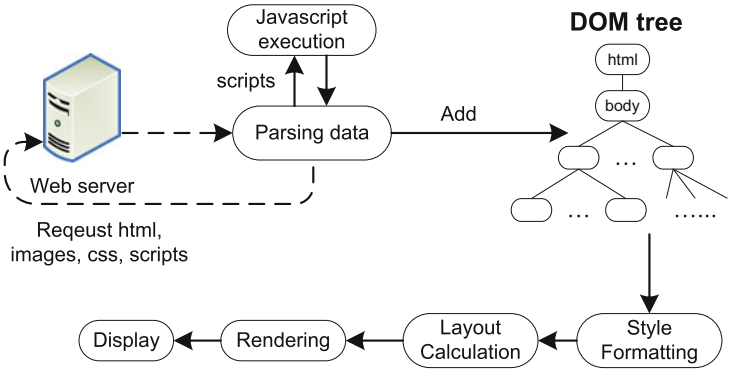

Fig. 3.1 The workflow of web page processing in smartphone based web browser

Figure 3.1 shows a simplified workflow of web page processing in web browsers. Loading an HTML page triggers a set of events: First, the HTML code is parsed. If there is JavaScript code, it is delivered to the interpreter engine for processing. If the web page includes URLs for objects like images, HTML files, and flashes, these objects are retrieved as well. Each added object becomes a node in the DOM tree. Next, to provide better user experience, the web browser processes the layout information, such as image decoding, style formatting, page layout calculation, and page rendering. Then a partially rendered display on the screen is drawn before completing the loading of the whole web page [31].

Due to limited computational power of smartphones, the time necessary to parse HTML, execute Javascript, build a DOM tree and finally transmit the layout of a page is expensive. In addition, due to the workflow for a browser's web page, the web page's loading time encompasses the entire distribution for transmissions. This loading time becomes much longer since the latency of wireless networks tends to be higher than that of wired networking. All of these factors add to the delay of web browsing on smartphones.

In 2G or 2.5G networks, due to the low network bandwidth, slow data transmission is the major reason for the long delays in web browsing. However, 3G networks have significantly high bandwidth (at least 100 KB/s in the current case), and consequently, the rate of data transmission should not be the major reason for long delays in web browsing. From various measurements, the CPU workload can easily reach 100 % for a long period of time during web browsing, and hence the local computing capability of smartphones (Android, iPhone, etc.) is the real bottleneck for opening most web pages.

Part of the reason for delays in opening web pages is their increasing complexity and their increasing population with objects. Since the web appeared in 1995, the number of objects per page has grown 21.7 times [32]. From 2003 to 2008, the number of objects in an average web page has nearly doubled from 25.7 to 49.9. Thus, opening modern web pages requires significant CPU power. Moreover, dynamic web

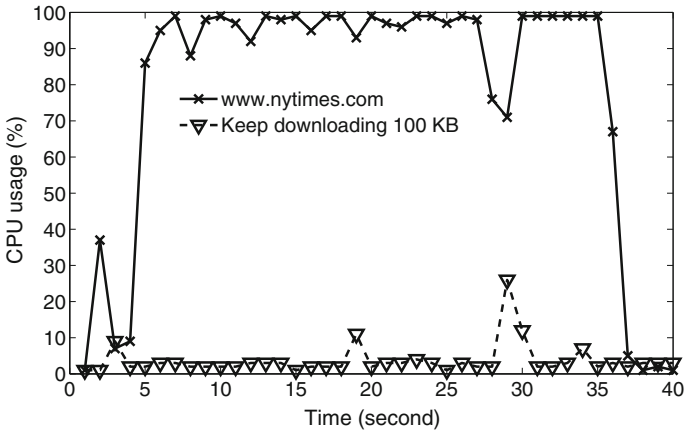

Fig. 3.2 The CPU usage of opening a web page or repeatedly downloading 100 KB of data

content from such sources as javascripts and flash, widely used in many web pages exacerbates the situation. Although desktop PCs can easily handle these dynamic web content, smartphone require significantly amounts of time and power to process javascripts and flash, to open these content rich web pages.

We measured the CPU usage of opening web pages on an Android smartphone (detailed experimental setup will be presented in Sect. 5.1), and the results are shown in Fig. 3.2. Figure 3.2 compares CPU usage on two cases: One for opening www.nytimes.com, and the other for continuous downloading of 100 KB of data. To download data, a developed small program establishes a socket connection between the smartphone and the server. The smartphone sends one byte of data to the server, which sends back 100 KB of data. After receiving this 100 KB, the smartphone repeats the process again. As shown in Fig. 3.2, to open a web page, the smartphone CPU workload reaches almost 100 % most of the time. After opening the web page (at time 37), the CPU workload declines to approximately 4 %. This Figure also shows that downloading data does not use an extraordinary amount of CPU capacity. Thus, the logical conclusion is that local computation related to web browsing is the bottleneck, which identifies key reason for long delays during web browsing.

Figure 3.3 further verifies this result. Since opening www.nytimes.com only requires downloading 522 KB of data, we investigate the amount of CPU capability the smartphone requires to download 522 KB of data. As shown in Fig 3.3, the CPU capacity needed is only 6 % and requires approximately 5.4 seconds, but the web browser requires more than 30 seconds to open the web page with the same amount of data.

To further verify that local computation is the bottleneck, we used a laptop and an Android smartphone to connect to the same WiFi AP, so that both accesses have the same bandwidth. To open the same web page, the smartphone took 30 seconds, while the laptop required only 5 seconds.

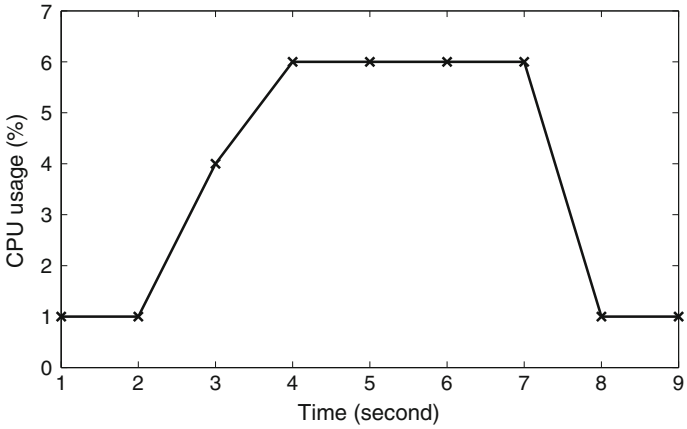

Fig. 3.3 The CPU usage for downloading 522 KB of data

3.2 Power Consumption of Web Browsing in 3G

The long delays and high CPU usage of web browsing also increase the power consumption of the smartphone. Figure 3.4 shows the power consumption of opening a web page and the power consumption of downloading 522 KB of data. As can be seen, using web browser will keep the power consumption at 1.08 W for a much longer time compared to downloading the same amount of data (522 KB) directly.

The long delays of opening a web page contribute to a significant part of the power consumption. Javascripts and flash are another part of power consumption after the web page is opened. This is because many of them keep consuming CPU

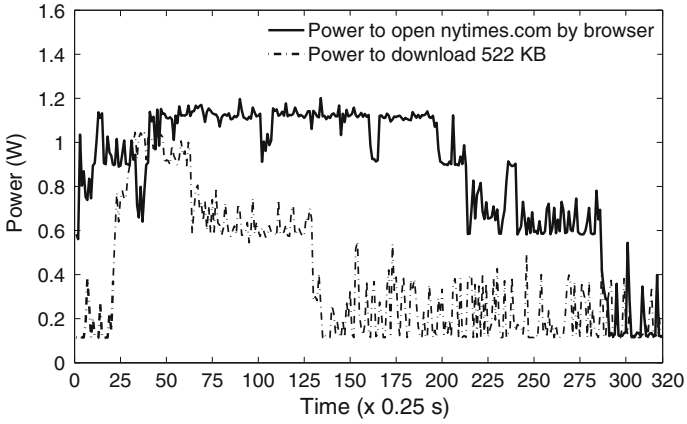

Fig. 3.4 The power consumption of web browsing and data downloading

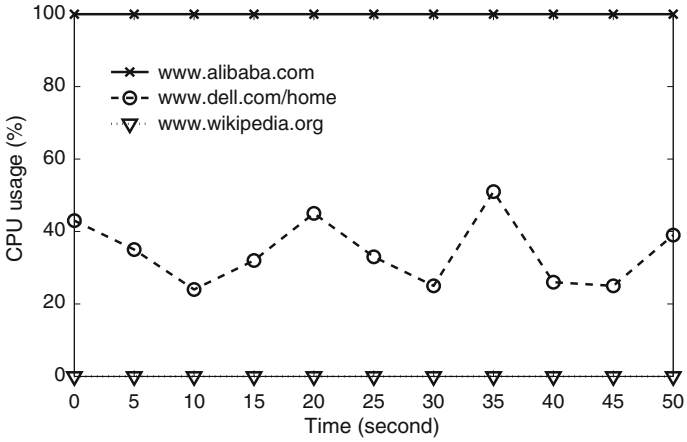

Fig. 3.5 The CPU usage after the web pages is opened on a smartphone

even if the user does not do anything. Figure 3.5 shows the CPU usage of a web browser on a smartphone after the web page has been completely opened. Because the web server "alibaba" has a lot of flash and other dynamic web content, the CPU utilization is very high. Although the "dell" web page looks static, there are javascripts running in the background, which keep consuming CPU. "wikipedia.org" does not have any dynamic content. After being opened, it does not consume any CPU. Although dynamic web content consumes most CPU cycles and battery power on the smartphone, there is little benefit to the user. Simply disabling these dynamic web contents may not work since some content would not be shown on the browser, and then affecting user experience.

Although the computing capability of smartphones is expected to increase in the future, web developers will design more complex web pages which consume more computational power. This motivates our design of the VMP architecture which aims to *shift* computing from smartphones to the cloud.

www.allitebooks.com

Chapter 4
Virtual Machine Based Proxy

Abstract This chapter presents detailed design of the Virtual Machine-based Proxy (VMP). We first give an overview of the VMP architecture, and then present efficient communication mechanisms based on compression and adaptation techniques. On the proxy side, we present resource management techniques to optimize the performance of the VMs and cloud techniques to address scalability issues. Security and privacy issues are addressed, and a technique based on trusted computing module is presented. Finally, we discuss deployment issues and issues on supporting interactive applications.

4.1 System Architecture

In the VMP based approach, the smartphone has to install an interaction module, which communicates with the proxy. With our VMP, a web request from the smartphone will be sent to the proxy which forwards the request to the web server and gets the reply from it. Then, the proxy sends the processed reply to the smartphone through the wireless network.

As discussed in the last chapter, to deal with the local computational bottleneck, a proxy is added to handle all the http requests, replies, and execute various java scripts and flashes. More specifically, on the proxy side, X11 [33] technique is used to display the web page on a "virtual" screen rather than a physical one. Then, a screen copy of the current web page is compressed with zlib [34] and transferred to the smartphone.

We have to implement an efficient communication mechanism for saving energy of the smartphone. This is because the communication mechanism of existing thin-client based approach is not efficient and consumes too much energy of the mobile devices. Existing thin-client based approaches are designed for PCs which have high speed Internet connection, and do not have power constraint. As a result, the primary goal of these approaches is to use increased network traffic to reduce the impacts

B. Zhao et al., *Mobile Web Browsing Using the Cloud*,
SpringerBriefs in Computer Science,
DOI: 10.1007/978-1-4614-8103-4_4, © The Author(s) 2014

of network delay on user experience. This is different from our application in 3G networks, where network latency is high. The round-trip time (RTT) over 3G is on the order of hundreds of milliseconds, or even seconds, while RTT over WiFi is at most 50 ms [30]. Thus, we have to design efficient communication mechanisms to address these issues.

In our VMP, by moving the browsing service to the proxy, the users' activities are visible to the proxy. Web access history or any other user-sensitive data is recorded in the browser's cache and other logs. Critical information such as digital certificates is also silently cached. Therefore, it is critical to isolate user information in the proxy to provide privacy and security protection. To achieve this goal, each smartphone uses its dedicated Virtual Machine (VM) on the proxy.

Figure 4.1 illustrates the overall architecture of our VMP. We assume Xen-based virtualization environment [35], and it will not limit the generality of the design. The big box represents a physical VM host, which contains a VM and Domain-0. Domain-0 is a specialized administrative VM that is responsible for managing hardware resources. All the I/O of the VMs must go through Domain-0 in the current Xen. Every physical host machine has one Domain-0 instance and there can be multiple guest VMs. There are three types of VMs in our design—*master VM*, *Web Cache VM*, and the *guest VM*. Guest VMs are used for actual computation offloading. Inside each VM, the Interaction module is responsible for the interaction between the VM and the client software on the smartphone. The master VM is the connection hub for the smartphone users. Users only need to know the address of this master VM which assigns a guest VM for a user. The VMP architecture can reduce the latency and network traffic of the smartphone. Although the data access delay between the

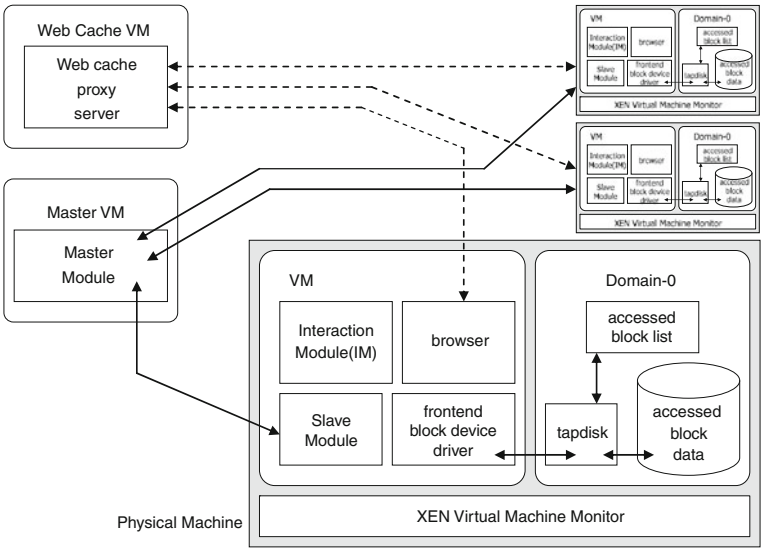

Fig. 4.1 The VMP architecture on the service provider

VMP and the web server is relatively low compared to the wireless part, there is a motivation to further reduce the traffic and latency between the web server and the VMP through caching; thus, a web cache VM is added in our VMP architecture so that the VMs can share the web cache.

4.2 Communication Mechanisms

We develop techniques to improve the energy efficiency of the communication mechanisms for opening web pages and for dealing with users' operations after opening the web page.

4.2.1 Features for Opening Web Pages

Instead of sending the characters in the URL one by one as in most thin-client based approach, the client only sends one URL request to the server and gets one response which includes the screen copy from the server. After getting the URL request, the server sends the screen copy after the web page is opened completely. Otherwise, some temporary incomplete content may be sent to the client who may not be interested in.

In addition, most web pages cannot be displayed in one screen due to the small display size of the smartphone. The original approach is to only send the current screen and send the next screen if the user requests. However, because of the long latency of 3G networks, it aggravates the delay experience on the user side, when they switch to the next screen. To eliminate this delay, we develop a multiple screen feature in which the server can send multiple screens of a web page to the client at once according to the user's request.

4.2.2 Supporting User Interactions

To enable the interactions between the user and the proxy, whenever the user moves the mouse, the client records the mouse position. When the position is on the boundary of the screen, or when the user clicks a link, the client sends this information to the proxy. Then, the proxy maps the mouse position at the client side to the position at the proxy side to open the web page, and sends the new screen display back. When the user moves the mouse to fill a text field on the web page, the client records the mouse position and the user input. After the user finishes, all data will be sent to the proxy which sends back the display feedback. To fill out multiple text fields, such as a table, each table content and its position are buffered and sent to the proxy together. To handle display update such as flash, the new data is only sent to the interested

user. Since the size of the smartphone screen is only one fourth of the screen at the proxy side, the proxy only needs to update that part of the screen. Thus, users still have web flash effect with our approach, but it does not consume any energy if the users are not interested in it.

4.2.3 Compressions

Due to the use of screen copy, there are many redundant information at the bit level. As a result, compression techniques at the proxy side are used to reduce the data size. Although the transmitted data size is reduced, the CPU time due to decompression at the smartphone side is increased. We look into different compression techniques such as MP3, JPEG, zlib, ... etc. to find out which one fits our application well, and we study the tradeoffs between reducing the data size and increasing the CPU workload on the smartphones in Sect. 5.3.3.

4.2.4 Adaptation Techniques

Based on the user location, the smartphone may get 3G service or downgrade to EDGE, etc. Even under 3G, depending on the signal strength, the data rate may vary widely. The VMP approach can also adapt to the wireless network condition of the smartphone if the smartphone includes the wireless network conditions in the web request. The proxy can reduce the resolution or only send text information if the user only has limited bandwidth, i.e., the user is inside a tunnel or far away from the base station.

4.3 Design of VMP at the Service Provider

The VMP system at the service provider interacts with the user handset software module to enable user web access in a power-efficient, high-performing and secure way. The key of the VMP at the side of the service provider is the cloud system that runs virtual machines to represent individual user handsets. This section describes the architecture of the VMP system and explains how each component interacts.

4.3.1 VM Management

We introduce three different type of VMs according to their roles—*Interaction VM*, *Master VM* and *Web Cache VM*. Each individual user of the VMP system is assigned

a dedicated *Interaction VM* for the duration of the service. The duration of service is determined by the user action. It is defined to be the time when the user requests the service start and the time when the user stops the web browsing. It is expected that there will be appropriate interface on the smartphone for these requirements. When a user first requests the service, a fresh *Interaction VM* has to be assigned. However, since the user cannot be expected to discover the address of the assigned *Interaction VM* we introduce the *Master VM* that serves as a kind of directory server. User's smartphone only maintains this single *Master VM*'s IP address and the first request is always directed to it. Specifically, the *Master VM* has two purposes.

- Assigning *Interaction VM* to new client: When a new client connects to the *Master VM*, it looks up the list of available VMs and assigns one of them to the client. The *Master VM*, thus, internally maintains the state of all the VMs that can be used for *computation shifting*. The *Interaction VM* can be in three states—*ready, offline, assigned*. In the *ready* state, the VM is awaiting for new assignment to the user. In the *offline* state, the VM is performing clean-up process as the previous user is done using it. In *assigned* state, the VM is currently assigned and serving a user.
- Initiating clean-up process: When the client finishes using the web and leaves the VM, the client-side IM notifies the *Master VM*. In turn, the Master Module (MM) of the *Master VM* commands the VM to restart. Underlying mechanism exists so that upon shut-down of the VM, all the modified disk blocks are reverted back to the original value. Detailed mechanisms for this part will be described in Sect. 4.3.2.

For the communication between the *Master VM* and the *Interaction VM*s, there exist a Slave Module (SM) software within the *Interaction VM* as a counter part of the MM (Master Module). They communicate over TCP/IP connection. Detection of client's entry into the system is easily done by accepting requests to VM assignment. Detection of client's departure requires a notification from the client since there is no clear event to use. Once the *Master VM* learns that a client has left, it initiates restart of the VM and updates the internal data structure for VM's state.

Figure 4.2 illustrate the sequence of steps when a client enters into the system and when it finishes using the VM and leaves. The client first contacts the Master VM and the Master VM assigns one idle VM's IP to the client. Then, the client tries again with the given IP address to make a connection. When the client leaves, it has to notify the Master VM of this event. Then, the Master VM does house-keeping of internal data structure and order the VM to reboot. As part of the rebooting process, the VM's image is restored to the initial state, clearing all trails of previous client. When the Master VM confirms that the VM is back online, it becomes ready for the next service. For this purpose, the *Interaction VM* sends a message to the MM of Master VM so that MM can switch the state of the VM from *offline* to *ready*.

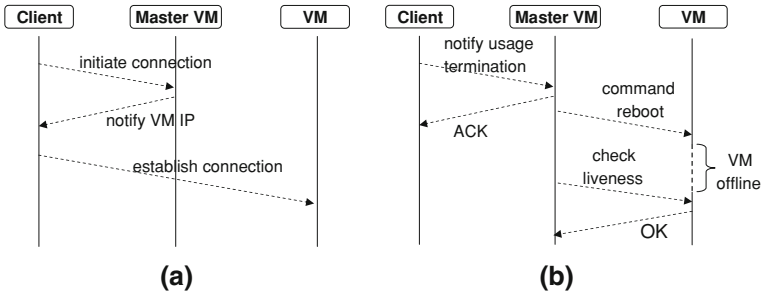

Fig. 4.2 Handshake sequence between Master VM and user VMs. **a** Client entry, **b** client departure

4.3.2 Managing VM Image

As users come and go through the VMP systems, *Interaction VMs* are assigned to different users. Although VMs inherently provide data security through isolation from other users while they are running, some of the user data left in the VM might be a possible security threat. If this VM is assigned to other users, sensitive data from previous users can be exposed. Therefore, we have designed our cloud computing cluster part of VMP system to completely eliminate such security threats. Our method is to start the *Interaction VM* from fresh VM image so that it is fundamentally impossible for other users to have any access to the previous user's data. One naive way to achieve this is to destroy the VM image when the client leaves and create a new one. However, because of VM image's size, it takes considerable amount of time to follow this procedure. If one VM image takes about 4 GBytes of space, creating new VM image from one master copy of VM image takes several minutes at the least. Instead, we maintain the list of modified blocks of the VM image while the user was using it. The original values of the modified blocks are appended and kept in a separate file. When the *Interaction VM* shuts down as the user leaves, modified blocks are overwritten to their original values.

4.3.3 Scalability Issues

Scalability of our VMP architecture is an important problem. In our design of the VMP system we can identify several points where scalability might be an issue. They are listed below and we also explain how we can achieve sufficient scalability for each of them. The first two aspects are concerned with responsive user experience whereas the third point is about the hard system limit which VMP can scale up to.

1. At the master VM: Since this is where the initial request is received, this can be a possible point of bottleneck. However, the primary job of the *master VM* is to

relay the request to the available Guest VM, the overhead of request handling is small. Conceptually the master VM is equivalent to the front-end web server that performs load-balancing to the next tier application servers in the multi-tier architecture and we have observed that modern server hardware can handle well over 1000 requests per second even at under 30 % of the CPU utilization. In case we need the capability to support high degree of scalability (e.g. over 1000 requests/s), we can consider two possibilities. We can consider introducing the tree hierarchy among the master VMs to further distribute the workloads. A simpler solution is to increase the capacity of the master VM. For example, we can switch the master VM instance type from small to large in case of Amazon EC2.

2. At the web cache VM: Similar to the master VM, since existing web cache software is well-tested to work well under heavy workloads, this node is unlikely to become a scalability hotspot.
3. Preparing Interaction VMs: Recall that in our current VMP system design, we dedicate individual VM to each requester. Although this provides good isolation property, it limits our scalability only up to the number of VMs we can simultaneously maintain. For example, if underlying VMP system can only support 100 VMs at a given time, we are only able to handle 100 request per second. If we consider other factors such as the VM preparation time and deprovisioning time, the scalability limit shrinks even further. We discuss this aspect in more details in the following.

VMP can achieve high scalability through leveraging the *scale-out* functionality provided by many of the current cloud services. *Scale-out* refers to the act of increasing the system capacity by adding additional computing unit to the system whereas *scale-up* means increasing the capacity of the existing computing units. Another term for scale-out and scale-up is horizontal scaling and vertical scaling, respectively. We can consider two scenarios in which VMP solution can be deployed. For the clarity of discussion, we assume here that the mobile service provider is the one who deploys the VMP solution in order to provide a value-added service to the subscriber. However, it is equally conceivable that any third party can launch the VMP solution.

- Deploying VMP in public cloud: In this scenario, mobile service providers builds VMP system on top of publically available clouds such as Amazon EC2, Rackspace or Windows Azure. The benefit is that they can utilize the auto-scaling feature that is the standard functionality in today's major public cloud. Details of the auto-scaling is described below. Depending on whether they use Infrastructure-as-a-Service (IaaS) cloud or Platform-as-a-Service (PaaS) cloud, the mobile service provider faces different set of tasks in deploying the VMP system. In case of using the IaaS cloud, they have to install all the middleware, supporting libraries and the VMP application. In case of PaaS, they have to rewrite the entire VMP application using the API exposed by the PaaS cloud provider. Also, the way in which the auto-scaling is used may differ from one cloud provider to another. This may require adapting the scaling function to the specific cloud.

- Deploying VMP in private cloud: Private cloud can be used when the mobile service provider owns and operates its own cloud infrastructure for internal use. This option has several advantages over using the public clouds. First, it can deliver better performance. Since private cloud can be tightly coupled with infrastructure for mobile networks by being physically adjacent to the VMP system, the performance of the VMP solution such as user-perceived response time can be better. Second, having more control over the underlying cloud infrastructure, it is possible to implement some features within the hypervisor layer. Recall from Sect. 4.3.2 where we described the mechanism of restoring VM image to its initial state by capturing all the modified disk blocks at the hypervisor layer. Such technique is executable when we have access to the hypervisor. Third, since the mobile service provider is making use of the infrastructure that already exists within the organization, this may translate to cost reduction. However, the downside is that the scaling function has to be implemented if not already existing in the private cloud.

4.3.3.1 Auto-Scaling of Current Public Clouds

Amazon EC2 cloud offers auto-scaling service by default. Users can specify the condition that triggers the auto-scaling actions from system metrics such as average CPU utilization, network usage or disk utilization. Values for these metrics are obtained for the Amazon EC2's monitoring service, called *Amazon CloudWatch*. *Amazon CloudWatch* offers seven default system metrics users can choose and users can additionally specify custom metrics.

Windows Azure also has the built-in auto-scaling function. Users are expected to specify the minimum and maximum number of instances as well as the conditions for triggering the actions as a part of configurations before deploying the cloud applications. Since Windows Azure is a PaaS cloud, users do not see the VM instance explicitly. Users only deal with what Windows Azure defines to be the roles. Role is a specific application format Windows Azure defines and is written with different APIs and code structures. For example, the *Web* role is expected to be the interface to the users and to relay the messages to the *Worker* role, which does not support graphical interfaces, but has a loop for job fetching. Although the abstraction of VMs are different from IaaS type of clouds such as Amazon EC2, Windows Azure also follows similar auto-scaling mechanisms.

There are public clouds without explicit auto-scaling functionality, as is the case with Rackspace. For those clouds, the third-party automation tools such as RightScale can be employed. These services add another layer to the bare public cloud and offers unified control of various auto-scaling behaviors.

4.3.3.2 Applying Auto-Scaling to VMP

So far we haven't discussed how many *Interaction VM* to be maintained in our VMP solution. For the best user experience (i.e. fastest response time of the initial

service request) the most ideal solution is to maintain the largest possible number of Interaction VMs up and ready for service. However, if we are using the public cloud the maintenance cost will be prohibitive since we will end up paying for the mostly idle VM instances. On the other hand, we cannot adopt the scheme of spawning VM instances upon the arrival of the requests since the VM provisioning time is typically at the order of minutes. Therefore, we create certain size of *Interaction VM* pool and dynamically grow or shrink this pool size depending on the volume of VMP service initiation requests.

The pool size can be determined by multiplying the VM provisioning delay d and the number of newly needed VM instances. We determine the number of newly needed VM instances as follows. Let us denote the request arrival rate as λ_t and the departure rate as μ_t at time interval $T_i < t < T_j$. Further we assume that the difference of these rates $\lambda_t - \mu_t$ follows the distribution $f(t : \theta)$ where θ is the distribution parameters. Denoting the percentile of f as t_p (i.e. t_p is the value such that $F(t_p) = p$ for $0 \leq p \leq 1$), we want to find t_p that minimizes the cost of using the public cloud and the average response time of user request. The value of p can be determined by:

$$p = \underset{0 \leq x \leq 1}{\operatorname{argmin}}\{t_x \cdot d \cdot c + g(t_x d)\} \tag{4.1}$$

where g describes the average response time as a function of the pool size.

In utilizing the auto-scaling function for VMP, we have to decide how to trigger the scaling actions. Since existing auto-scaling function provided by the public clouds are based on the system metrics such as CPU utilization or network usage, those are not applicable to VMP. We want to trigger the auto-scaling whenever the pool size changes from previous time window. The pool size calculation can be done within the Master VM and it should also invoke the auto-scaling API for pool size adjustments instead of utilizing the commonly used system metrics.

4.3.4 Implementation Issues

In the implementation of our prototype, we utilize Xen's *blktap* mechanism [36]. When the *Interaction VM*'s web browser generates block requests (i.e. read or write of storage blocks), they are first received by the *front-end* block device of the VM after going through the VFS layer and ext2 file system layer as well as buffer cache. In the *para-virtualized* Xen virtual machine, the *front-end* block device driver has a corresponding *back-end* block device driver. They transfer blocks through shared memory called *ring buffers*. As the block requests arrive to the *back-end* block device driver within the Domain-0, they are issued to the real block device driver in Domain-0. In case of read request, requested data blocks are transferred back to the *Interaction VM* in the reverse order. Xen's *blktap* mechnism replaces the *back-end* driver with *blktap* driver and allows it to be controlled in the user space of Domain-0. In the *blktap* mechanism, there are two user-level processes. One is the

blktapctrl process that manages the *tapdisk* process. The *tapdisk* process is where the VM's block requests are delivered. It is created and killed by the *blktapctrl* process whenever the block device is attached to or detached from the VM. The *blktapctrl* process creates different type of *tapdisk* process according to the configuration files. Different type of *tapdisk* process differs in how to handle the incoming block requests. Among several supported types, there are *synchronous* and *asynchronous* types. If the *tapdisk* process is using the *synchronous* mode, all the incoming block requests are handled in a synchronous manner. (i.e. blocking read and write calls are used to process requests one by one.) In the VMP implementation, we use the *asynchronous* type of *tapdisk*.

The *tapdisk* process provides a convenient location to intercept VM-generated block requests and perform some actions. We intercept all the write requests here and maintain the list of all the modified blocks with their original value. We have created the *accessed block list* and *accessed block data* file. This list is updated whenever the tapdisk process receives write requests. The tapdisk process defines a callback interface for the *close* event. When the user handset's SM notifies the MM, MM orders the *Interaction VM* to reboot. As part of the rebooting process, we have added a function to restore all the modified blocks to their original values. *Asynchronous* I/O is used to undo the modification of blocks. When the VM comes back online ready to serve other users, all the VM's contents are restored to clean initial values without any trails of previous users.

4.4 Security and Privacy Issues

Our proxy based architecture provides mobiles users significantly better web browsing experience with improved energy efficiency as well as reduced delay; however, it also comes with security and privacy issues that we need to solve. Although the VM based approach can address most security and privacy issues, some applications such as banking access may require more security and privacy. As our service shifts mobile users' workload to the proxy, all HTTP requests are initiated from the proxy, which communicates directly with the end web servers in different HTTP session. Then, mobile users' secrete information like password cannot be sent in hashed form as used in traditional secure HTTP protocol (https) [37, 38]. It needs to be readable by our proxy in order to initiate a HTTP request. On the other hand, it cannot be sent in pain text; otherwise the traffic sniffers would learn or modify the content of the mobile users' requests. Similar problems exist when the response from the proxy is sent back to the end user. More importantly, the transaction inside https should not be logged or viewed to protect user privacy. To ensure confidentiality and integrity, a secure communication protocol between the proxy and the end user is necessary.

www.allitebooks.com

4.4.1 Security-Enhanced Model for the VMP Architecture

Here we introduce a secure model for our VMP architecture to address the aforementioned security and privacy issues. Our goal is to support services provided by https, which ensures confidentiality and integrity of communication between the VMP server and mobile users, without changing any existing transportation protocol involving network operators and end web servers. Note that this security model is only used for secure protocols such as https or when the user has explicit requests. For most of other HTTP requests, this security model is not necessary.

Figure 4.3 illustrates our VMP with trusted computing platform, which consists of User Information database, Request Scheduler, individual Virtual Machines (VMs), trusted virtual machine monitor (TVMM), and Trusted Computing module. The mobile user sends an encrypted request to the proxy, who can use the Trusted Computing module to decrypt the request and encrypt the response. The User Information database keeps user-specific configuration information required for answering requests for each user. For example, it stores the browser preference information for each mobile user. Also, it keeps the public key for each user, as well as the certificate that binds the public key to the user's identifier. The public key will be retrieved by the Encrypting module to encrypt the response message. The Request Scheduler module receives users' requests and forwards them to VMs for processing. It can make a schedule for each request based on user's priority or frequency. When processing is finished, the Request Scheduler module returns the response generated by the VMs and sends it back to the user.

The Trusted Computing module might become the bottleneck of the entire proxy-based service, since every request and response needs to go through the Trusted Computing module. To address this problem, we develop a parallel processing system by uniting this with a trusted virtual machine monitor (TVMM). That is, a high-assurance virtual machine monitor that partitions a single tamper-resistant, general-purpose platform into multiple isolated VMs. Using the TVMM, existing applications and operating systems can each run in a standard VM. In this way, each incoming

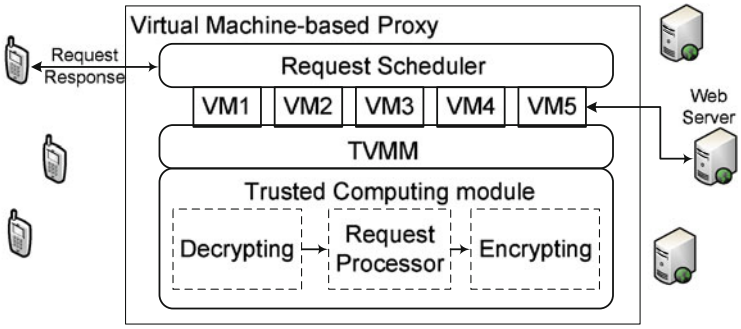

Fig. 4.3 Security-enhanced model for the VMP architecture

request is processed by individual VMs built above the Trusted Computing module in parallel. An example system Terra [39] has used this model and built a VM-based platform for trusted computing.

The Trusted Computing module includes three components: Decrypting, Request Processor, and Encrypting. The dotted line of the three components means that their operations are actually performed at VMs but monitored and protected by the Trusted Computing module. The decrypting module decrypts user's request message and verifies the signature of the message. The Request Processor module accepts the decrypted request from the Decrypting module, and processes this request by communicating with end web servers. It then forwards the results to the Encrypting module. The encrypting module is able to retrieve each user's public key from User Information database without letting the proxy be aware of which database entries are retrieved [40]. It then signs and encrypts the response message and forwards it to the Request Scheduler. All these operations are actually performed by individual VMs, which are built above and monitored by the Trusted Computing module, without proxy's interference.

4.4.2 Asymmetric Encryption and Authentication System

Here we use asymmetric encryption and authentication schemes [41] through out our secure model, that is, the sender and receiver use a pair of public and private keys to ensure secure communication.

In our secure communication protocol, each message needs to be encrypted [42–45] and digitally signed [46–48] before it can be transmitted. First, the proxy generates a pair of keys and stores the private key in sealed storage in its Trusted Computing module. The public key is distributed to each mobile client who is registered with the service. When a mobile client registers with the proxy, it generates another pair of keys. The private key is kept secretly in smartphone (i.e. in kernel) while the public key is distributed to the proxy, who stores the public keys of each client in its User Information database, associated with each client's identifier. According to Public Key Infrastructure (PKI) [49–52], the Trusted Computing module retrieves a certificate from certificate authority (CA) that binds the client's public key to its identifier, and stores the certificate in the User Information database of the proxy.

We assume mobile users are powerful enough to perform asymmetric cryptographic operations, such as decrypting ciphertexts or signing messages. With public-key cryptosystem, the sender first digitally signs the message with its private key, encrypts the message along with the signature with receiver's public key and then sends the message to the receiver. After receiving the message, the receiver uses his private key to decrypt this message, and verify the signature as well, to make sure this message has not been eavesdropped or modified.

For further security consideration, we can assume each end maintains a key pool with multiple keys in it. It signs all messages with the same private key within a given time interval, after which it switches to a new key without any user intervention.

This prevents an attacker from compromising the client and learning the current key. Changing keys periodically ensures privacy. Exampling encryption/decryption methods include ECC [53], RSA [54, 55] and so on.

4.4.3 Trusted Computing Module

The Trusted Computing module [56, 57] accepts message from the Request Scheduler module and processes users' requests. The module has two main properties. First, it is possible for a user to remotely ensure that the module can access the user's secrete information in each request only if the software running on the module corresponds to a configuration approved by the user (or a third-party auditor on user's behalf). Second, the proxy deploying the Trusted Computing module cannot learn the request that is being processed by the module. [58] uses a similar concept to enhance privacy in Location-Based Service scenario. Examples of Trusted Computing module architecture for user privacy can be exploited from [59, 60].

We can implement our module with classic Trusted Platform Module (TPM), as suggested by the Trusted Computing Group (TCG) [61]. TPM is implemented with remote attestation and sealed storage to guarantee a trusted computing environment [62]. Remote attestation lets an entity verify whether the software (including operating system and applications) running on a remote machine corresponds to an expected configuration. Sealed storage prevents certain encrypted information from being decrypted on a machine unless the software running on the machine corresponds to a given configuration. In our model, we achieve these two requirements in the following way: each user creates an asymmetric key pair and gives the public key to the proxy, which stores the key in the User Information database. The encrypting module retrieves this public key to encrypt the user's response when being queried by the proxy. Each user gives his public key to the Trusted Computing module only when this user (or a third-party auditor on the users behalf) approves the software configuration of the module. This approval reveals the concept of remote attestation. On the other hand, our Trusted Computing module generates a second asymmetric key pair for each mobile user and gives the public key to the user, who uses the public key to encrypt his request and send to the proxy. To avoid the private key being leaked upon compromise of the Trusted Computing Module, it keeps the key in sealed storage in the Trusted Computing module. The decrypting module could use the private key to decrypt user's request and send it to Request Processor for processing. In this way, if the module gets compromised and its software configuration is changed by an intruder, the private key becomes inaccessible and the module can no longer decrypt the user's request.

To further ensure that the proxy deploying the Trusted Computing module cannot learn users' secrete information from this module, or to prevent malicious attackers from physically compromising the machine, we must make sure the information should never be logged. Moreover, it should be immediately erased after its usage to decrease the risk of information leakage [63, 64]. A user (or a third-party auditor

on his behalf) needs to review the software configuration running in the Trusted Computing module. This software includes the operating system and algorithms that are required by proxy-based service. We should try to keep the Trusted Computing module as simple as possible, which makes user easier to review [65] . In this sense, BIND [66], a software service that runs on a TPM enabled machine and provides fine-grained attestation, can be used to implement our enhanced Trusted Computing.

4.4.4 Antivirus Scanning Service

Recent research [29, 67, 68] shows that there is an increasing number of malware such as worm and virus in smartphones. When mobile users access the web site with malware, the malicious files are downloaded to the smartphones. Then, these malwares can monitor user activities, steal user information or take control of the smartphone. Virus scan is one of the most common functionalities provided by most antivirus software. In virus scan, the scanner reads files, compares them with some virus signatures, and returns a result of "virus found" or "not" to the user. Obviously, the scan process is CPU and I/O intensive. In case of smartphones, performing such virus scans adds delay and increase energy consumption.

In our VMP architecture, an antivirus scanner can be deployed as a service on the proxy. When a user sends a request to the proxy for opening a web page, the proxy will open it first and scan the downloaded files. The feasibility of building an antivirus service outside of protected VM has been proved in the VMwatcher [69] which can be deployed directly in our VMP.

Smartphone users can benefit from our antivirus scanning service in the following way: first, it saves battery power on smartphones; second, as the CPU and I/O intensive workloads are shifted to the proxy, the scanning will not affect the common usage of the smartphone; finally, the scan in the proxy will be much faster than that on the resource limited smartphone.

4.4.5 System Overhead

The current smartphones are powerful enough to perform asymmetric cryptographic operations, such as decrypting ciphertexts or signing messages. The security communication overhead between proxy and smartphone is negligible since we use public-key based cryptosystem. Most of the public-key cryptographic systems are small [70, 71], typically in the order of tens of bytes (using Elliptic Curve Cryptography, or RSA) [72], and can be easily implemented [73]. Our security communication protocol adds no more overhead than original HTTPS/SSL protocol, but provides a communication environment with confidentiality and integrity between proxy and smartphones, which cannot be achieved by using HTTPS/SSL protocol. The overhead is negligible comparing with the size of the transmitted message, especially when

the message is large. We can optimize this by aggregating the user inputs before they are sent out, as explained in Sect. 4.2.

The cost of implementing the Trusted Computing module is obvious since it is a hardware-based solution for security. However, it is non-replaceable in our proxy-based service where users' private information is involved [74]. Other software-based security solutions are vulnerable to various attacks thus cannot assure our security requirements, more importantly, cannot win users' trust in using our service. Different versions of Trusted Platform module have been widely deployed from large centralized servers to personal computers [60, 75]. Also, the techniques are well developed and implemented across multiple platforms including Linux, Mac OS and Windows [39, 76, 77]. The decreasing cost of the Trusted Computing module is easily overweighed by the requirement of a seamless security environment.

As mentioned before, our asymmetric cryptosystem and Trusted Computing module would only need to be activated when sensitive information is involved in transmission (when https protocol is used). In most cases, users do not send out sensitive information and also do not require secure communication between themselves and web servers. Then, these requests would be transmitted in plaintext and processed by VMs without involving the Trusted Computing module to save the computation and transmission overhead.

4.5 Discussions

4.5.1 Supporting Interactive Applications

Our VMP approach can significantly reduce the power consumption and delay of various web browsing, even for highly interactive applications. If the interactive application frequently communicates with the Web server such as Google map, VMP should perform better since it is similar to sending multiple web browsing requests and getting multiple replies. If the application only interacts locally with the user without frequently accessing the Web server, there are two cases. (1) the application is based on computationally intensive flash or javascript such as games. However, this is not common for smartphones since the user can download the game and play locally. Even if this is done online, VMP may still outperform conventional approaches since the CPU power consumption is very high for these computationally intensive flashes using the original browser. Another option is to record the web sites that the VMP approach does not perform well and bypass the proxy for these web sites. (2) Another kind of interactive application may not consume too much CPU such as filling out table forms. In our solution, after the user moves the mouse to a text field (table entry) and clicks it, the client pops out a window for the user to input the text, and it records the mouse position and the user input. After the user fills out all the fields, the client sends the recorded mouse position and user inputs to the proxy which will

fill out the forms. To enhance user experience, the client can also repeat this process after each field is filled out.

4.5.2 Deployment Issues

There are different ways to deploy the VMP system. For example, the service providers such as T-Mobile, AT&T, and Verizon can use the existing public cloud computing service to build the VMP system. Although this approach has low cost, public cloud may not meet the security level, and the network traffic between the cloud and the 3G network may introduce extra overhead. It is better for these service providers to build their own private cloud, which can be located closer to the 3G infrastructure to reduce the network traffic. Also, the service provider can provide other value added service later if they have the private cloud. There are other ways to adopt the VMP architecture at the initial stage. For example, some early adopters can run the proxy on their office (or home) computers. An organization can run a proxy to serve employees inside that organization or paid users. Building VMP on top of cloud computing can take care of the scalability issue because one of the main benefits of cloud is the ability to automatically scale based on the workload.

Chapter 5
Performance Measurements

Abstract In this chapter, we evaluate the performance of our VMP approach, and compare it to other approaches under various system settings. We first describe the experimental setup and the web page benchmark used for evaluation. Then, we evaluate the delays and power consumption of web browsing for different approaches. For delay, we consider the effects of wireless signal strength, screen resolution, and data compression. For power consumption, we evaluate the effects of various factors such as the web page size, the cache effect, and the CPU utilization after the web page is opened.

5.1 The Experimental Setup

In our experiment, Android Dev Phone 1 with Android 1.5 is used, and it connects to the Internet through T-Mobile 3G UMTS network. We use a Linux PC at the proxy side which runs VMP with Xen 3.1.4. Each VM runs Ubuntu 8.04 Desktop OS. The proxy is connected to the Internet and can be accessed by the smartphone through T-Mobile 3G UMTS network.

Figure 5.1 shows the experimental setup. To accurately measure the power consumption of the smartphone, we use Agilent E3631A Power Supply to provide the current with constant voltage (3.706 V) to the smartphone instead of using the battery. The Agilent E3631A connects to our laptop through the "NI GPIB-USB-HS" cable. We install LabVIEW on the laptop, and use it to program the Agilent E3631A to capture the current of the smartphone every 0.25 seconds. Then, we can easily calculate the power consumption of the smartphone.

B. Zhao et al., *Mobile Web Browsing Using the Cloud*,
SpringerBriefs in Computer Science,
DOI: 10.1007/978-1-4614-8103-4_5, © The Author(s) 2014

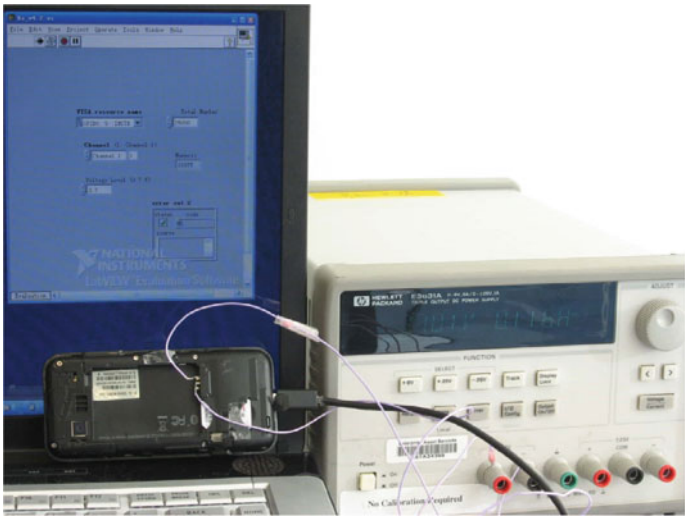

Fig. 5.1 The experimental setup to measure the power consumption of the smartphone

5.2 The Web Page Benchmark

Our web page benchmark consists of 20 popular web pages, which are selected from the top popular web pages listed in the Alexa website [78]. To have a fair comparison between the VMP approach and the normal web browser, the selected web pages that meet the following two requirements. First, it should be correctly rendered by the smartphone browser. We will not use web pages that cannot be displayed correctly by the smartphone browser because of lack of software support. Secondly, we will not use the web pages that provide different versions specifically for mobile devices. For these web page, the actual content will be different when they are opened by the VMP from when opened by mobile devices. Finally, if the same website has different language versions, such as "Google" websites, we only use the most common language version. The cumulative distributions of the web page size and the CPU usage after being opened are shown in Figs. 5.2 and 5.3, respectively. The average data size of these web pages is 483 KB, and the average CPU usage after being opened is 31.7 %.

We cannot select too many web pages in our benchmark due to the limitation of the cache size of the Android browser. The maximum cache size is only 8 MB. To evaluate the performance of the web browser with cache, we have to limit the number of web pages to make sure that there is enough cache space to hold them. We will not use existing benchmarks such as SPECweb2009 [79] because they are developed for evaluating web server performance and because they are outdated as is the case for the i-Bench [80] which was developed in 2000.

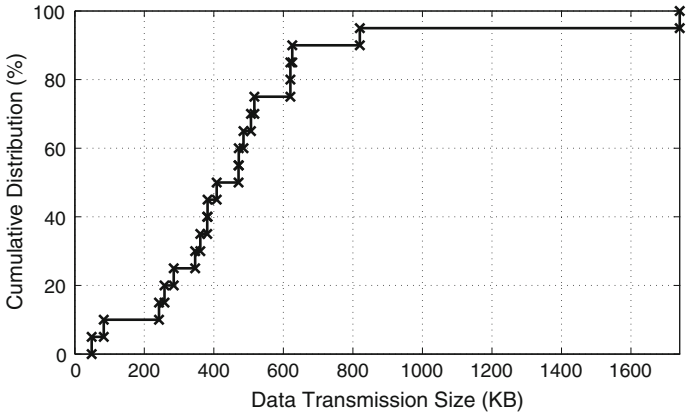

Fig. 5.2 The cumulative distribution of the data transmission size of benchmark web pages

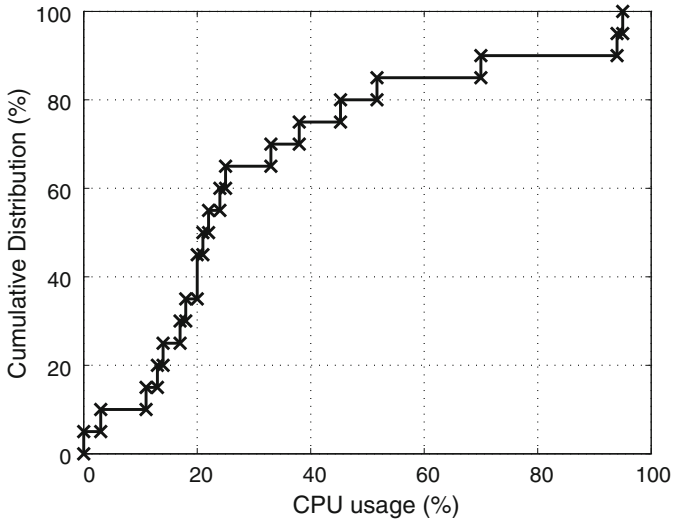

Fig. 5.3 The cumulative distribution of the CPU usage after opening the benchmark web pages

5.3 Experimental Results on Delay

In this section, we first present the average delay for opening benchmark web pages, and then explain the reasons behind it. We also evaluate the effects of compression and transmission of multiple screen copies on delay.

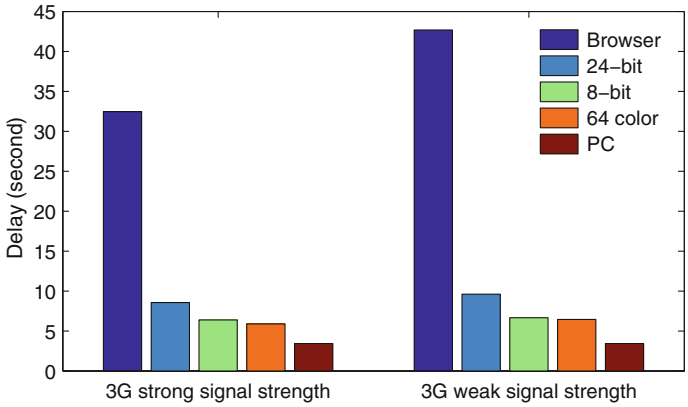

Fig. 5.4 The average delay for opening benchmark web pages

5.3.1 Average Delay for Opening Benchmark Web Pages

In this section, we evaluate how much delay the VMP approach can reduce, compared to the normal smartphone browser. In VMP, the delay for opening a web page consists of three parts: the delay to send the request to the proxy, the delay to open the web page at the proxy, and the delay to receive the data from the proxy. When the proxy sends the screen copy of the web page to the smartphone, there are different resolutions to choose: 64 color, 8-bit (256 color), 16-bit, 24-bit.

Figure 5.4 shows the average delay of accessing the benchmark web pages. It compares five cases: the normal smartphone browser, the normal PC, and three cases of VMP: 64 color, 8-bit, and 24-bit. These comparisons are under two different environments: strong signal strength and weak signal strength. To get strong signal, we use the smartphone near the base station. For weak signal, we use the smartphone in our office where the signal strength indicator shows minimal value. Because the signal strength affects the data transmission rate, it generally takes longer time under weak signal to download the same amount of data.

As shown in Fig. 5.4, using PC to access the web has the lowest delay, using the normal smartphone browser has the highest delay, and the VMP approach is in the middle. Comparing three VMP approaches, the 64 color approach has lower delay than the 8-bit approach which is lower than the 24-bit approach. Intuitively, the resolution of 24-bit will be much clear than the 64 color, but it needs to transmit much more data. Note that the delay of VMP includes the delay for the proxy to open the web page which is almost equal to the delay of the PC approach. After removing this part of delay, the delay of the 24-bit approach is much higher than that of the 64-color approach due to transmitting much more data. Since the smartphone screen is very small, the difference of different approaches is not that clear, and the 8-bit approach looks good. Therefore, we choose the 8-bit resolution for our VMP. From

www.allitebooks.com

Table 5.1 Open web pages with VMP and the normal web browser

	VMP	Browser
Wiki delay (s)	3	6
nytimes delay (s)	5.9	41
Wiki data transmission (KB)	109	83.3
nytimes data transmission (KB)	493.5	522

Fig. 5.4, we can see that our VMP (8-bit) approach can reduce the delay by 80 % compared to normal smartphone browser.

To understand the long delays for web browsing, we need to look into the details. We study two cases: one is to access "www.wikipedia.org" which has 83.3 KB of data, and the other one is to access "www.nytimes.com" which has 522 KB of data. Table 5.1 shows the delay and data transmission size for opening these two web pages with VMP and the normal smartphone browser.

As can be seen from Table 5.1, using the smartphone browser has much longer delay when accessing "nytimes" compared to VMP. As explained in Chap. 3, the CPU usage of opening "nytimes" with smartphone browser reaches almost 100 % most of the time. It also shows that the local computation related to web browsing is the bottleneck, which is the key reason for the long delays in web browsing. On the other hand, the VMP approach does not have the local computation bottleneck, and hence it has much lower delay to open the web page.

From Table 5.1,we can also see that the delay of accessing "www.wikipedia.org" which is very simple and small, is still longer than the VMP approach. This can be explained by Fig. 5.5 which shows the CPU usage for opening wiki by smartphone browser and by VMP. As shown in the figure, at time 3, the button is clicked to open the web page, and some CPU time is used to handle the screen touch action. The smartphone browser uses much more CPU power to open the web page. It takes 6 seconds to open the web page which includes time to execute various stages of the http protocol and handle some local computation. The VMP approach only takes 3 seconds to open the web page.

We also measure the delay of accessing some typical web pages. Figure 5.6 shows the results of accessing three web pages: "www.wikipedia.org", "espn.go.com/sports", and "www.dell.com/home". In small web page such as "www.wikipedia.org", VMP only has a small advantage compared to the normal smartphone browser. However, for web pages which have many javascripts and flashes, VMP can significantly reduce the delay, for example, reducing the delay by 86 % in opening "espn.go.com/sports" and reducing the delay by 92 % in opening"www.dell.com/home". As explained earlier, the VMP approach shifts the computation to the proxy, and hence reduces the delay.

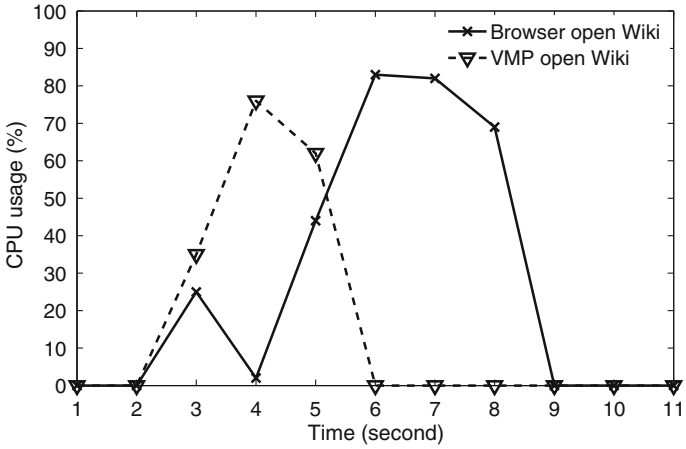

Fig. 5.5 The CPU usage for opening Wiki

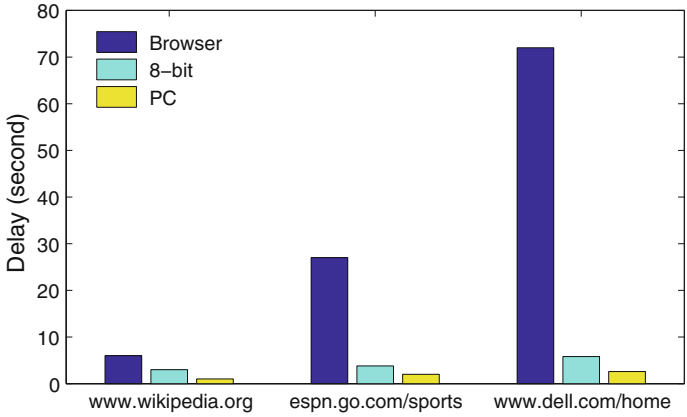

Fig. 5.6 The delay of extreme cases with strong 3G signal

5.3.2 Effects of Transmitting Multiple Screens of Data

Since most web pages cannot be displayed in one screen, we evaluate the delay of transmitting multiple screens of data to the smartphone. Figure 5.7 shows that compared to the delay of transmitting one screen of data, the delay for transmitting two screens of data only slightly increases for about 0.6–0.8 seconds. In VMP, transmitting multiple screens of data only increases the delay from receiving the data from the proxy. The data size for transmitting an additional screen of data is only about 70 KB. Because the smartphone bandwidth with 3G connection is around 100 KB/s, the delay increase for transmitting multiple screens of data is very small.

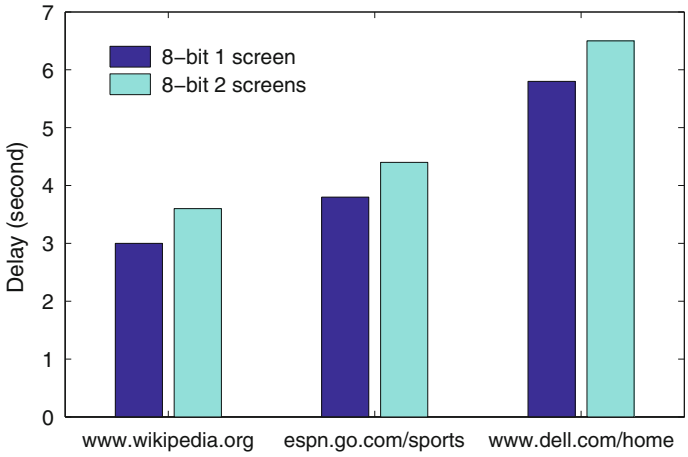

Fig. 5.7 The delay for transmitting multiple pages together

Table 5.2 Open Wiki with and without compression

	With compression	Without compression
Delay (s)	3	16
Data transmission (KB)	109	1460

5.3.3 Effects of Data Compression

VMP uses zlib to compress the display data at the proxy side to reduce the data transmission size. However, the client has to spend a little bit more power on computing to decompress the data. Thus, we evaluate the delay with and without data compression to find out if we also need to remove this computing. Table 5.2, shows that even for a small web page such as wiki, without data compression, the data size increases from 109 to 1460 KB, which adds about 13 seconds of delay for a 100 KB/s link. Since the screen copy has lots of redundancy, the data size can be significantly reduced by compression. Also, the decompression function is very simple, and it does not consume too much CPU time on the smartphone. Thus, it is easy to see the advantage of keeping decompression computing on smartphone.

For a web browsing service, if the delay for opening a web page is too long, users may lose patience and give up before it is opened. Therefore, it is also very important to limit the delay range. Figure 5.8 shows that the delay range with the normal web browser changes dramatically according to the web pages being accessed. The red square is the average delay value. Because the delay range with PC is small and the data transmission latency is nearly constant, the delay with VMP is within a small range. For example, with 8-bit in VMP, users can open any web page within 10 seconds.

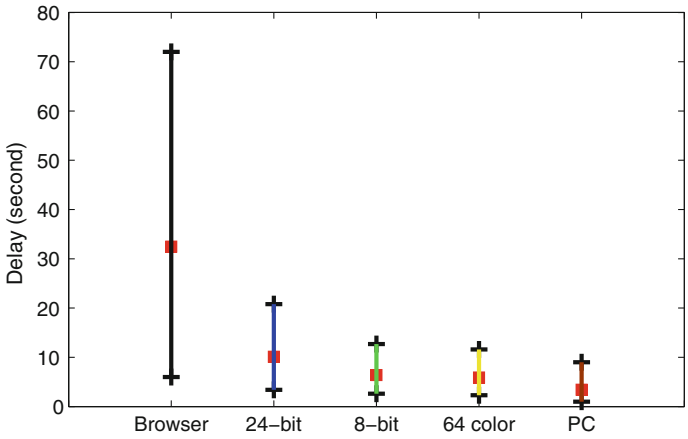

Fig. 5.8 The delay range with strong signal

Table 5.3 The average delay of opening benchmark web pages with Android and iPhone

	Android	iPhone
Browser without cache (s)	32.465	39.1
Browser with cache (s)	26	29.85
CPU (MHz)	528	620
Memory (MB)	192	128

Finally, we want to study if the high access delay of Android browser is caused by device specific design and implementation. We use iPhone 2.0 to open the same benchmark web pages and measure the average delay with strong signal. Table 5.3 shows that the average delay of Android is slightly less than that of iPhone for opening the same benchmark web page with similar hardware configuration. This indicates that, for most 3G smartphones, the high delay of web browsing may be independent of their browser design and implementation, and it is due to the limited computational capability of the smartphone. Therefore, shifting the computation from the smartphone is a good solution to address the limited computation capability and to reduce the delay and power consumption of the smartphone.

5.4 Experimental Results on Power Consumption

To understand the power consumption of smartphones, we have performed several experiments and the results will be presented in this section.

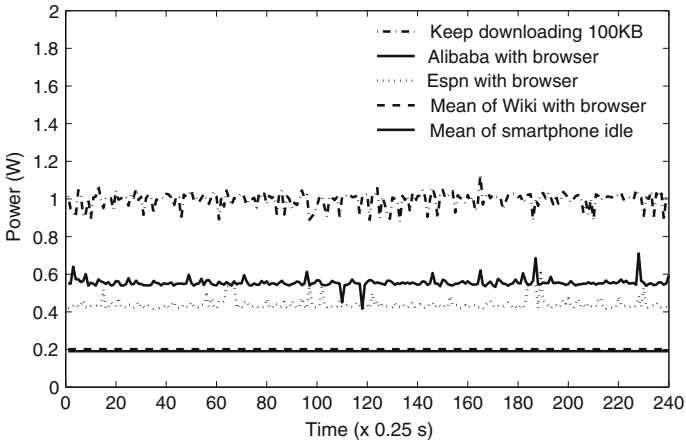

Fig. 5.9 The power consumption of typical web pages after being opened

5.4.1 Power Consumption of the Smartphone Browser

In this experiment, we show the power consumption of the smartphone browser with the following three typical web pages.

- Wikipedia.org: small size web page; needs to transmit 83 KB data; will not consume any CPU power after being opened.
- espn.go.com/sports: medium size web page; needs to transmit 421 KB data; keeps using 19 % CPU power after being opened.
- alibaba.com: large size web page; needs to transmit 514 KB data; keeps using 100 % CPU power after being opened.

In addition to these three cases, we also consider results with two extreme cases for comparison: smartphone in *idle* state, and in heavy downloading. Note that *idle* state is different from the *sleep* state. For the heavy downloading case, we use "keep downloading 100 KB" described in Chap. 3.

As shown in Fig. 5.9, data transmission consumes the largest amount of power, which is about 1 W. After downloading the website "alibaba", the local CPU is 100 % saturated, and it consumes about 0.6 W. For "ESPN", which consumes about 19 % CPU after the web page is opened, the power consumption is about 0.44 W. The power consumption of "wiki" and idle has the lowest power consumption, which is about 0.2 W. From Fig. 5.9, we can see that repeated downloading of data consumes the largest amount of power, and keeping the CPU running also consumes a lot of power compared to keeping it idle.

Figures 5.10 and 5.11 compares the power consumption between VMP and normal smartphone browser when opening web pages. For opening "espn.go.com/sports", the power consumption of VMP drops at time 60, since it does not need to transmit any data after that. However, the normal smartphone browser only finishes downloading

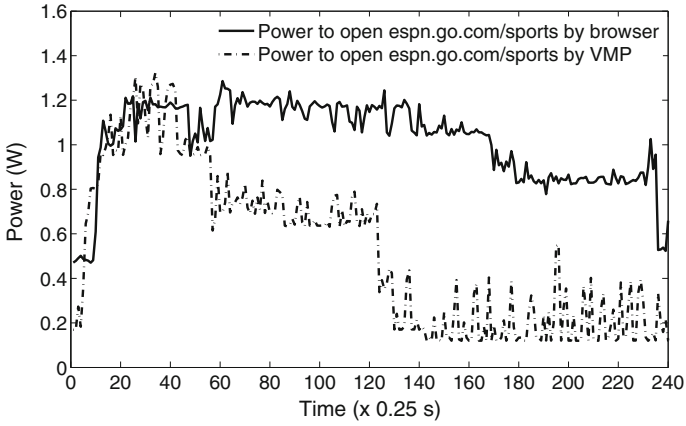

Fig. 5.10 The power consumption for opening ESPN sport

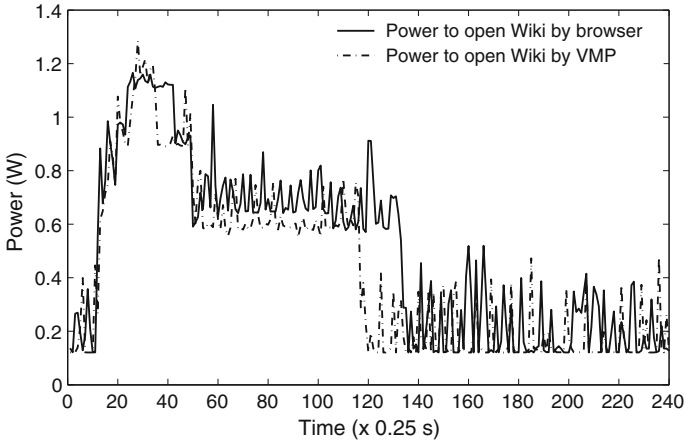

Fig. 5.11 The power consumption for opening Wiki

data at time 160. For opening "wiki", which only has a small amount of data. The power consumption of both approaches has similar pattern. The only difference is that VMP finishes downloading a little earlier as explained before (see Fig. 5.5) Thus, the power consumption of VMP drops at time 120 whereas that of the normal smartphone browser drops at time 140.

Next, we evaluate the power consumption of VMP and browser for two extreme cases: "wiki" (CPU usage 0 %) and "alibaba" (CPU usage 100 %). We repeatedly open the same web page with certain interval for an hour to get an idea of the power consumption in a more general setting. We have two cases "with cache" and "without cache". To get the results of "without cache", the web cache and cookie of the browser are erased before being opened next time. This is achieved by modifying the browser

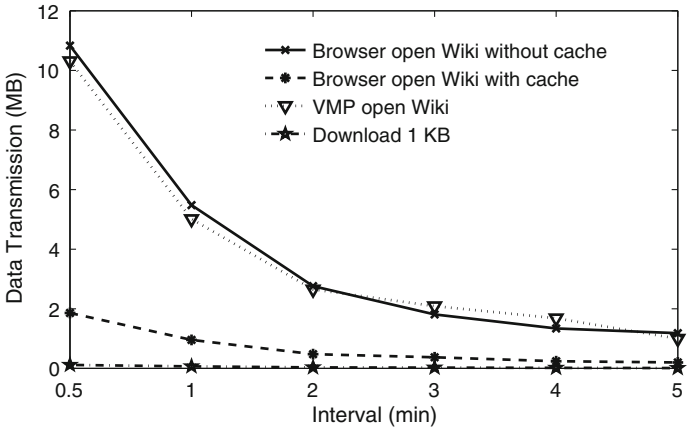

Fig. 5.12 The amount of data transmitted when repeatedly open Wiki with different time interval for an hour

of Android to automatically do the test and erase the cache before each web access. To compare with the browser approach fairly, VMP send all screen copies of the web page to the smartphone.

Figure 5.12 compares the amount of data transmitted in the following four cases: *browser with cache, browser without cache*, VMP, and *repeatedly downloading 1 KB of data*. This *1 KB case* will provide a lower bound on the amount of data transmitted. The 1 KB case transmits the smallest amount of data, and the browser with cache has the next lowest amount of data. The other two have similar amount of data. As the time interval increases, the amount of transmitted data decreases since the phone spends more time in the idle stage.

Figure 5.13 compares the power consumption of these four cases. As expected, the 1 KB case provides the lower bound for power consumption. However, the power consumption of the four cases is similar although they transmit different amount of data. The power consumption is mainly decided by the time to open it. As shown in Table 5.1, VMP needs about 3 seconds open wiki, similar to "wiki with cache". The delay of "wiki without cache" is about 6 seconds. This explains why the VMP approach has lower power consumption than "wiki without cache".

Figure 5.14 compares the power consumption of these four cases when opening web page "alibaba". This time, when the interval is 0.5 min, the VMP approach cuts the power consumption by 25 % compared to the browser approach. As explained before, the power consumption is related to the delay to open it. The delay to open "alibaba" by the browser without cache is 35 seconds, while the delay with cache is 24 seconds. The delay to open it by the VMP approach is 7 seconds. Although "alibaba" with cache downloads much less data compared to VMP (as shown in Fig. 5.15), it still consumes much more power because the delay to get the data is much longer than the VMP approach due to heavy local computations.

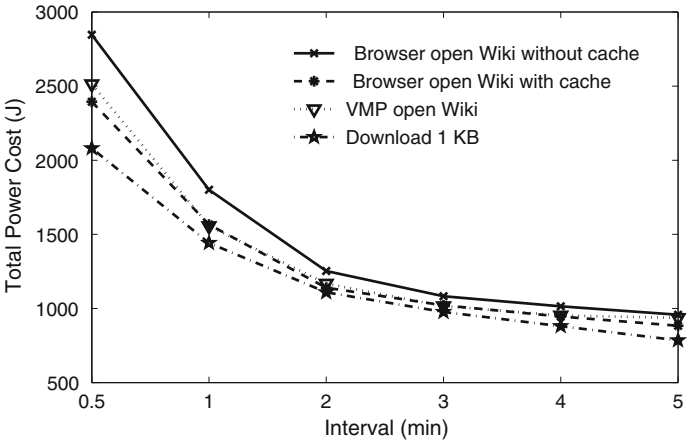

Fig. 5.13 The amount of power consumed when repeatedly open Wiki with different time interval for an hour

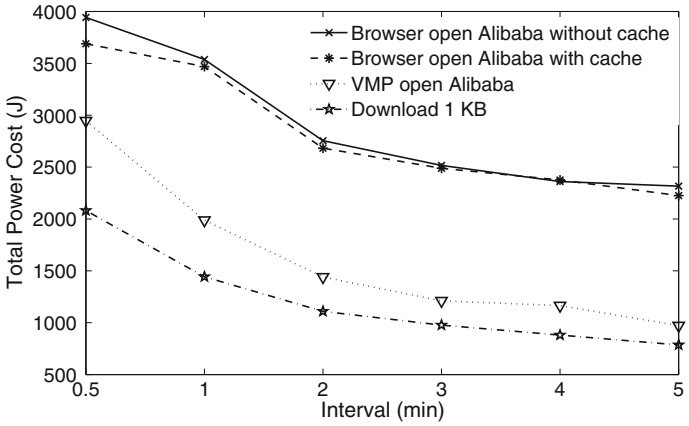

Fig. 5.14 The amount of power consumed when repeatedly open Alibaba with different time interval for an hour

As the time interval increases to 5 min, the data downloading time becomes much smaller compared to the idle time. Thus, the power consumption between VMP and the 1 KB case becomes closer since both have the same power consumption during idle. The VMP approach reduces the power consumption by about 58 % compared to the browser approach. This is because the power consumption is mainly determined by the CPU utilization when the time interval is long. Since "alibaba" still utilizes 100 % CPU after the browser opens the web page, its power consumption is much higher than the VMP approach which is idle most of the time. Note that there is no difference between "cache" and "without cache" since both need 100 % CPU.

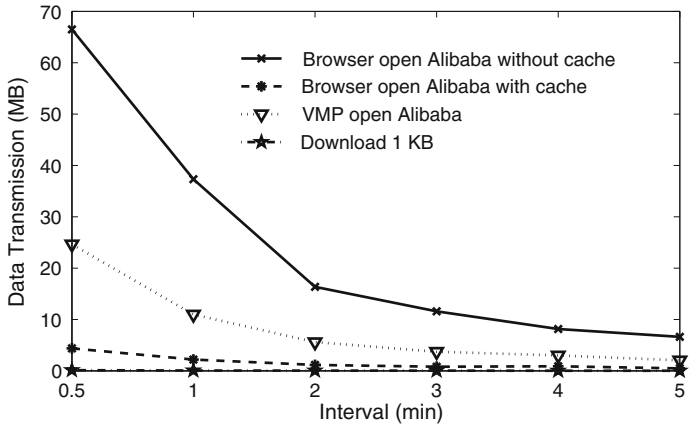

Fig. 5.15 The amount of data transmitted when repeatedly open Alibaba with different time interval for an hour

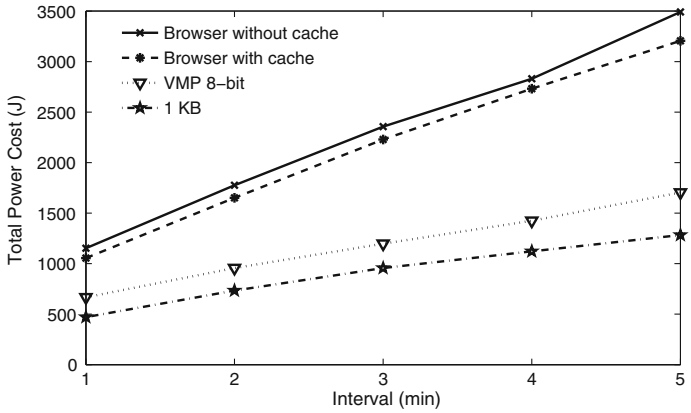

Fig. 5.16 Power consumption with strong signal

5.4.2 Power Consumption with Web Page Benchmark

In this section, we use the benchmark web pages to compare the power consumption of those four cases. In this experiment, instead of opening the same web page, the smartphone opens each web page in the benchmark and stays for certain time interval, then opens another web page until all 20 web pages have been opened.

Figure 5.16 compares the power consumption under strong signal, and Fig. 5.17 compares the amount of data transmitted. Since only 20 web pages are opened, the amount of transmitted data does not change as the time interval increases. However, the amount of power consumed will increase as the time interval increases because the phone will spend more time in the idle stage which also consumes power.

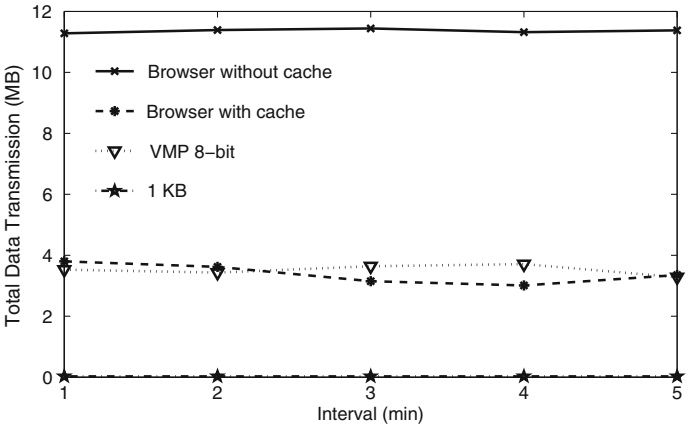

Fig. 5.17 Data transmission with strong signal

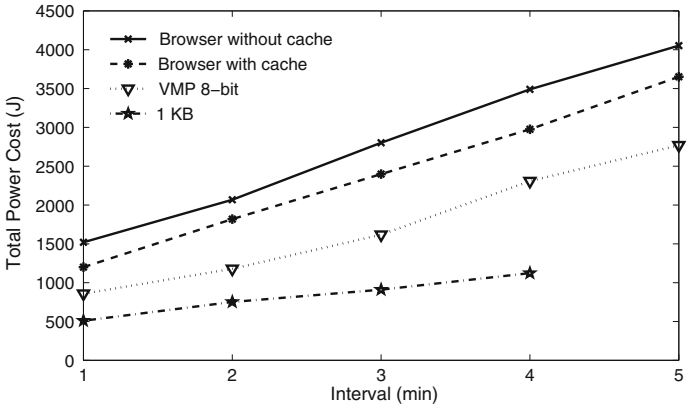

Fig. 5.18 Power consumption with weak signal

The results are generally consistent with the results in Sect. 5.1. Although "Browser with cache" can reduce the amount of data transmitted, the power consumption is still close to "Browser without cache" because it takes similar amount of computing to open the web page and the same CPU utilization after the web page is opened. However, compared to using the normal browser, VMP can reduce the power consumption by about 45 %, and it is close to the lower bound provided by the 1 KB case.

Figures 5.18 and 5.19 compare the power consumption and the amount of data transmitted when opening the benchmark web pages under weak signal. The major difference between these results from those under strong signal is that more power is consumed under weak signal. This is because weak signal will reduce the data transmission rate and increase the transmission time, and hence consume more power.

www.allitebooks.com

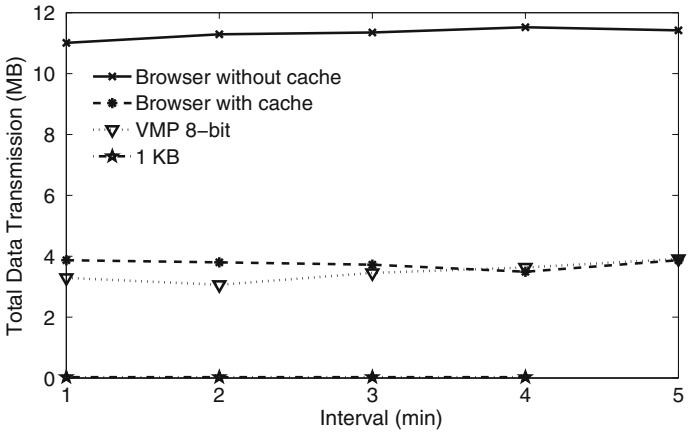

Fig. 5.19 Data transmission with weak signal

Table 5.4 The cost of performing antivirus scanning in terms of time and power consumption

Folder size (MB)	SMobile on smartphone		Symantec on PC (VMP)
	Time (s)	Battery (%)	Time (s)
10	120	2	12
50	600	3	31
100	1140	6	57
200	2340	10	110

When the signal strength is very weak, if the client does not have any data transmission with the server for a long time, the connection will be lost. That is why we cannot obtain the power consumption for the "1 KB case" with 5 min interval.

5.4.3 The Benefit of Performing Antivirus Scanning with VMP

To show the benefit of antivirus scanning service in the VMP approach, we install SMobile VirusGuard [81] on the smartphone and use it to scan folders with different sizes. All folders are filled with Android Package (APK) files which are used for application installation. As shown in Table 5.4, the time and battery (energy) needed for scan increases as the folder size increases. When the folder size is 200 MB, it takes more than 39 min and 10 % battery to complete the scan. During the scan, the user will feel the slowdown when using the smartphone. Table 5.4 shows much less time is needed for scanning the same folders on VMP using Symantec Endpoint Protection software. Since PCs are in the cloud and connected to the power source, power consumption is not shown.

Chapter 6
Conclusions and Future Research Directions

Abstract We conclude the book in this chapter. Based on our work, we have identified the key reason of the long delays and high power consumption in web browsing, and have presented the Virtual Machine-based Proxy (VMP) to address the problem. Experimental results validate the benefits of the VMP approach. Finally, we point out future research directions.

In this book, we have discovered that the key reason of the long delays and high power consumption in web browsing is not due to the bandwidth limitation most of time in 3G networks. The local computational limitation at the smartphone is the bottleneck for opening most web pages, especially for those with rich dynamic content such as javascripts and flash. To address this issue, we have designed an architecture, called VMP, to shift the computing from smartphones to the VMP. With our VMP, a web request from the smartphone will be sent to the proxy which forwards the request to the web server and gets the reply from it. Then, the proxy sends the reply to the smartphone through the wireless network. Instead of sending the original HTTP reply which contains many CPU intensive objects, the proxy simply sends a processed screen copy to the smartphone.

The book also includes cloud techniques to address scalability issues, resource management techniques to optimize the performance of the VMs on the proxy side, compression techniques to further reduce the bandwidth consumption, and adaptation techniques to address poor network conditions at the smartphone. We have also addressed security, privacy, and real deployment issues.

The book illustrates the feasibility of deploying the proposed VMP system in 3G networks through a prototype using Xen virtual machines (in cloud) and Android Phones with ATT UMTS network. Experimental results show that compared to normal smartphone browser, our VMP approach reduces the delay by more than 80 % and reduces the power consumption during web browsing by more than 45 %.

There are some limitations with our prototype for small size web pages. Some simple web pages may not have the dynamic web content, and then the size of the web page may be pretty small. In such cases, doing a screen copy many not save any data

B. Zhao et al., *Mobile Web Browsing Using the Cloud*,
SpringerBriefs in Computer Science,
DOI: 10.1007/978-1-4614-8103-4_6, © The Author(s) 2014

transmission. Thus, if the data size of the web is smaller than a threshold, the proxy can switch to a *bypass* mode, where the data is sent to the client directly without doing screen copy. Furthermore, the client side can maintain a web page list, which collects the web pages whose sizes are smaller than a threshold. For these web pages, the web access requests will be sent to the web servers directly instead of going through the proxy. We want to emphasize that small data size web pages are limited. Although our proxy based solution may send a little bit more data for these web pages, the access delay is negligible since the transmitted data is limited. Also, the compression techniques may solve part of the problem, and will be further studied. Although we have discussed issues related to supporting interactive applications, security, privacy, and real deployment, these issues still deserve further investigation.

Acknowledgments This work was supported in part by the National Science Foundation under grant CNS-1218597.

References

1. The magic moment:smartphones now half of all U.S. mobiles, venturebeat.com, (2012–03-29)
2. R. Powers, Batteries for low power electronics. Proc. IEEE **83**(4), 687–693 (1995)
3. L. Zhong, N.K. Jha, Energy efficiency of handheld computer interfaces: limits, characterization and practice, in *Proceedings of the Third ACM International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2005
4. J. Flinn, M. Satyanarayanan, Managing battery lifetime with energy-aware adaptation. ACM Trans. Comput. Syst. **22**(2), 137–179 (2004)
5. J. Liu, L. Zhong, Micro power management of active 802.11 interfaces, in *Proceeding of the 6th international conference on Mobile systems, applications, and services (MobiSys)*, pp. 146–159, 2008
6. R. Han, P. Bhagwat, R. LaMaire, T. Mummert, V. Perret, J. Rubas, Dynamic adaptation in an image transcoding proxy for mobile web browsing. IEEE Pers. Commun. **5**(6), 8–17 (1998)
7. T.W. Bickmore, B.N. Schilit, Digestor: device-independent access to the World Wide Web. Comput. Netw. ISDN Syst. **29**(8–13), 1075–1082 (1997)
8. A. Moshchuk, S.D. Gribble, H.M. Levy, Flashproxy: transparently enabling rich web content via remote execution, in *Proceeding of the 6th International Conference on Mobile Systems, Applications, and Services (MobiSys)*, pp. 81–93, 2008
9. Opera mini, http://www.opera.com/mobile/features/ (2010)
10. S. Iyer, L. Luo, R. Mayo, P. Ranganathan, Energy-adaptive display system designs for future mobile environments, in *MobiSys '03: Proceedings of the 1st International Conference on Mobile Systems, Applications and Services* (ACM, New York, 2003), pp. 245–258
11. J. Sorber, N. Banerjee, M.D. Corner, S. Rollins, Turducken: hierarchical power management for mobile devices, in textitMobiSys '05: Proceedings of the 3rd International Conference on Mobile Systems, Applications, and Services (ACM, New York, 2005), pp. 261–274
12. N. Kamijoh, T. Inoue, C.M. Olsen, M.T. Raghunath, C. Narayanaswami, Energy trade-offs in the IBM wristwatch computer, in *ISWC '01: Proceedings of the 5th IEEE International Symposium on Wearable Computers* (IEEE Computer Society, Washington, DC, 2001), p. 133
13. P.J. Havinga, G.J. Smit, Design techniques for low-power systems. J. Syst. Archit. **46**(1), 1–21 (2000)
14. R.A. Baratto, L.N. Kim, J. Nieh, THINC: a virtual display architecture for thin-client computing. SIGOPS Oper. Syst. Rev. **39**(5), 277–290 (2005)
15. B.K. Schmidt, M.S. Lam, J.D. Northcutt, The interactive performance of SLIM: a stateless, thin-client architecture. SIGOPS Oper. Syst. Rev. **33**(5), 32–47 (1999)
16. B.C. Cumberland, G. Carius, A. Muir, Microsoft Windows NT Server 4.0, Terminal Server Edition: Technical Reference, Microsoft Press Std
17. B. Harvey, Virtual network computing, Linux J., p. 5
18. D. Converse, J. Gettys, A. Mento, R. Scheifler, *X Window System: Core and extension protocols*, ser. ISBN 1-55558-148-X. Digital Press, 1996, vol. releases 6 and 6.1

B. Zhao et al., *Mobile Web Browsing Using the Cloud*,
SpringerBriefs in Computer Science,
DOI: 10.1007/978-1-4614-8103-4, © The Author(s) 2014

19. A.M. Lai, J. Nieh, B. Bohra, V. Nandikonda, A.P. Surana, S. Varshneya, Improving web browsing performance on wireless pdas using thin-client computing, in *WWW '04: Proceedings of the 13th International Conference on World Wide Web* (ACM, New York, 2004), pp. 143–154
20. J. Kim, R.A. Baratto, J. Nieh, pTHINC: a thin-client architecture for mobile wireless web, in *WWW '06: Proceedings of the 15th International Conference on World Wide Web* (ACM, New York, 2006), pp. 143–152
21. A. Fox, E.A. Brewer, Reducing WWW latency and bandwidth requirements by real-time distillation, in *Proceedings of the Fifth International World Wide Web Conference on Computer Networks and ISDN Systems* (Elsevier Science Publishers B. V., Amsterdam, 1996), pp. 1445–1456
22. E. de Lara, R. Kumar, D.S. Wallach, W. Zwaenepoel, Collaboration and multimedia authoring on mobile devices, in *MobiSys '03: Proceedings of the 1st International Conference on Mobile Systems, Applications and Services* (ACM, New York, 2003), pp. 287–301
23. J. Smith, R. Mohan, C.-S. Li, Content-based transcoding of images in the internet, in *Proceedings of 1998 International Conference on Image Processing, ICIP 98*, Oct 1998, vol. 3, pp. 7–11
24. I. Ahmad, X. Wei, Y. Sun, Y.-Q. Zhang, Video transcoding: an overview of various techniques and research issues. IEEE Trans. Multimedia **7**(5), 793–804 (Oct. 2005)
25. Amazon kindle. [Online]. Available: http://www.amazon.com/kindlefamily
26. B.-G. Chun, P. Maniatis, Augmented smart phone applications through clone cloud execution, in *HotOS XII: Proceedings of the 12th Workshop on Hot Topics in Operating Systems*, May 2009
27. M. Satyanarayanan, P. Bahl, R. Caceres, N. Davies, The case for vm-based cloudlets in mobile computing. IEEE Pervasive Comput. **8**(4), 14–23 (2009)
28. B. Zhao, B.C. Tak, G. Cao, Reducing the delay and power consumption of web browsing on smartphones in 3G networks, in *Proceedings of the 22nd IEEE International Conference on Distributed, Computing Systems (ICDCS)*, 2011
29. B. Zhao, Z. Xu, C. Chi, S. Zhu, G. Cao, Mirroring smartphones for good: a feasibility study, in *International Conference on Mobile and Ubiquitous Systems(Mobiquitous)*, 2010
30. E. Cuervoy, A. Balasubramanianz, D.-k. Cho, A. Wolmanx, S. Saroiux, R. Chandrax, P. Bahl, Maui: making smartphones last longer with code offload, in *Proceedings of the 1st International Conference on Mobile Systems, applications and Services (MobiSys)* (ACM, New York, 2010)
31. L.A. Meyerovich, R. Bodik, Fast and parallel web page layout, in *Proceedings of the 19th International Conference on World Wide Web (WWW)*, 2010, pp. 711–720
32. A.B. King, Average web page size triples since 2003, http://www.websiteoptimization.com/speed/tweak/average-web-page
33. T. Richardson, F. Bennett, G. Mapp, A. Hopper, Teleporting in an x window system environment. IEEE Pers. Commun. **1**(3), 6–12 (1994)
34. P. Deutsch, Zlib compressed data format specification, RFC 1950, IETF
35. P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, A. Warfield, Xen and the art of virtualization, in *SOSP '03: Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles* (ACM, New York, 2003), pp. 164–177
36. Xen blktap, http://wiki.xensource.com/xenwiki/blktap
37. E. Rescorla, HTTP over TLS, IETF, 2000
38. A. Freier, P. Karlton, P. Kocher, SSL 3.0 specification, http://wp.netscape.com/eng/ssl3, 1996
39. T. Garfinkel, B. Pfaff, J. Chow, M. Rosenblum, D. Boneh, Terra: a virtual machine-based platform for trusted computing. ACM SIGOPS Oper. Syst. Rev. **37**(5), 206 (2003)
40. E.K. B. Chor, O. Goldreich, M. Sudan, Private information retrieval, in *Proceedings of 36th IEEE Symposium on Foundations of Computer Science*, October 1995
41. C. Grecas, S. Maniatis, I. Venieris, Introduction of the asymmetric cryptography in GSM, GPRS, UMTS, and its public key infrastructure integration. Mob. Netw. Appl. **8**(2), 145–150 (2003)
42. J. Jonsson, B. Kaliski, Public-key cryptography standards (PKCS), Springer-Verlag, ISBN: 3-540-44007-0, 2003

43. R. Merkle, Protocols for public key cryptosystems, in *IEEE Symposium on Security and Privacy*, 1980, pp. 122–134
44. A. Salomaa, *Public-Key Cryptography* (Springer, Berlin, 1996)
45. W. Kuchlin, Public key encryption. ACM SIGSAM Bull. **21**(3), 69–73 (1987)
46. FIPS PUB 186–3, Digital Signature Standard (DSS), Gaithersburg, MD, June, 2009
47. R. Rivest, A. Shamir, L. Adleman, *On Digital Signatures and Public-Key Cryptosystems* (Laboratory for Computer Science, Massachusetts Institute of Technology, 1977)
48. D. Denning, Digital signatures with RSA and other public-key cryptosystems. Commun. ACM **27**(4), 392 (1984)
49. R. Housley, W. Ford, W. Polk, D. Solo, Internet X. 509 public key infrastructure certificate and CRL profile, 1999
50. C. Adams, S. Lloyd, *Understanding Public-Key Infrastructure: Concepts, Standards, and Deployment Considerations* (Macmillan Technical Publishing, Indianapolis, 1999)
51. H. Li, Y. Wang, Public-key infrastructure, *Payment Technologies for E-commerce*, (Springer, Berlin, 2003), pp. 39–70
52. U. Maurer, *Modelling a public-key infrastructure* (Lecture Notes in Computer Science, Springer, 1996)
53. N. Koblitz, Elliptic curve cryptosystems. Math. Comput. **48**(177), 203–209 (1987)
54. L. Adleman, R. Rivest, A. Shamir, A method for obtaining digital signature and public-key cryptosystems. Commun. ACM **21**(2), 120–126 (1978)
55. H. Williams, A modification of the RSA public-key encryption procedure (Corresp.). IEEE Trans. Inform. Theory **26**(6), 726–729 (1980)
56. S. Pearson, B. Balacheff, *Trusted Computing Platforms: TCPA Technology in Context* (Prentice Hall PTR, New Jersey, 2002)
57. C. Mitchell et al., Trusted Computing, *Trusted computing*, IET, 2005
58. U. Hengartner, Enhancing user privacy in location-based services, Centre for Applied Cryptographic Research, University of Waterloo, Tech. Rep. CACR, vol. 27, 2006
59. R. Sailer, X. Zhang, T. Jaeger, and L. Van Doorn, Design and implementation of a TCG-based integrity measurement architecture, in Proceedings of the 13th USENIX Security, Symposium, 2004, pp. 223–238.
60. A. Iliev, S. Smith, D. Coll, N. Hanover, Protecting client privacy with trusted computing at the server. IEEE Secur. Priv. **3**(2), 20–28 (2005)
61. Trusted computing group, https://www.trustedcomputinggroup.org
62. T. Module, Specifications, https://www.trustedcomputinggroup.org/specs/TPM, 2007
63. J. Camenisch, Better privacy for trusted computing platforms. Lect. Notes Comput. Sci. **3193**, 73–88 (2004)
64. T. Eisenbarth, T. Guneysu, C. Paar, A. Sadeghi, D. Schellekens, M. Wolf, Reconfigurable trusted computing in hardware, in *Proceedings of the ACM Workshop on Scalable Trusted Computing*, 2007
65. S. Smith, *Trusted Computing Platforms: Design and Applications* (Springer, New York, 2005)
66. E. Shi, A. Perrig, L. Van Doorn, BIND: a time-of-use attestation service for secure distributed systems, in *Proceedings of IEEE Symposium on Security and Privacy*, 2005
67. Z. Zhu, G. Cao, S. Zhu, S. Ranjan, A. Nucci, A social network based patching scheme for worm containment in cellular networks, in *Proceedings of IEEE INFOCOM*, Rio de Janeiro, Brazil, April 2009
68. B. Zhao, C. Chi, W. Gao, S. Zhu, G. Cao, A chain reaction dos attack on 3g networks: analysis and defenses, in *Proceedings of IEEE INFOCOM*, Rio de Janeiro, Brazil, April 2009
69. X. Jiang, X. Wang, D. Xu, Stealthy malware detection through vmm-based out-of-the-box semantic view reconstruction, in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, 2007, pp. 128–138
70. J. Camenisch, A. Lysyanskaya, A signature scheme with efficient protocols, *Lecture Notes in Computer Science*, 2003, pp. 268–289
71. T. ElGamal, A public key cryptosystem and a signature scheme based on discrete logarithms. IEEE Trans. Inform. Theory **31**(4), 469–472 (1985)

72. E. Fujisaki, T. Okamoto, How to enhance the security of public-key encryption at minimum cost. Lect. Notes Comput. Sci. **1560**, 53–68 (1999)
73. P. Barrett, Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor, in *Advances in Cryptology-CRYPTO*, vol. 86. Springer, 1987, pp. 311–323
74. E. Felten, Understanding trusted computing: will its benefits outweigh its drawbacks? IEEE Secur. Priv. **1**(3), 60–62 (2003)
75. D. Molnar, A. Soppera, D. Wagner, Privacy for RFID through trusted computing, in *Proceedings of ACM Workshop on Privacy in the Electronic Society*, 2005
76. B. Kauer, OSLO: improving the security of trusted computing, in *Proceedings of the USENIX Security Symposium*, 2007
77. J. Rushby, A trusted computing base for embedded systems, in *Proceedings of the Seventh DoD/NBS Computer Security Initiative Conference*, 1984, pp. 294–311
78. A. I. Inc. (2009) The top 500 sites on the web. http://www.alexa.com/topsites
79. Specweb 2009. http://www.spec.org/web2009/
80. Z.-D. Inc. (2000) i-Bench version 1.5. http://etestinglabs.com/benchmarks/i-bench/ibench.asp
81. S. S. Inc. (2009) Smobile virusguard for google android. http://www.smobilesystems.com

www.allitebooks.com